Applied Soft Computing Journal 190 (2026) 114589

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Dynamic graph-enhanced multi-scale attention network for FW-UAV fault diagnosis under small sample

Rong Yi, Jianguang Lu*, Xianghong Tang

State Key Laboratory of Public Big Data, Guizhou University, Guiyang, 550025, China

HIGHLIGHTS

- Propose a dynamic graph-enhanced mechanism for structural representation learning.
- Design a multi-scale attention module to capture complex temporal dynamics.
- An adaptive feature aggregation strategy to enhance feature complementarity.

ARTICLE INFO

Keywords:
FW-UAV
Fault diagnosis
Dynamic graph enhancement
Multi-scale attention
Feature aggregation

ABSTRACT

Deep learning-based fault diagnosis has achieved significant success in intelligent industrial monitoring. However, diagnosing faults in fixed-wing unmanned aerial vehicles (FW-UAVs) under limited data conditions remains challenging because of scarce fault data, complex spatiotemporal dynamics of flight signals, and difficulty in simultaneously capturing global dependencies and local topology evolution patterns. To address these challenges, we propose a novel dynamic graph-enhanced multi-scale attention network (DGMANet) for robust fault diagnosis of FW-UAVs under small-sample conditions. Specifically, DGMANet integrates a dynamic graph enhancement mechanism that models localized topological dependencies and adaptive inter-feature correlations, thereby enabling effective structural feature learning even with limited data. A multi-scale attention module is incorporated to extract hierarchical global dependencies, which permits the model to concentrate on salient temporal patterns across multiple resolutions. Furthermore, an adaptive feature aggregation strategy is introduced to fuse local and global representations, enhancing feature complementarity while suppressing noise and redundancy. Extensive experiments on four real-world flight datasets demonstrate that DGMANet consistently outperforms existing baselines in small-sample scenarios, validating its effectiveness and generalization capability in resource-constrained UAV monitoring scenarios.

1. Introduction

In recent years, driven by rapid advances in unmanned aerial vehicle (UAV) technology, UAVs have been widely adopted in environmental monitoring [1], intelligent transportation [2], agriculture [3], and intelligent control technology [4] owing to their structural flexibility and cost-effectiveness. UAVs currently represent a primary research focus. Nonetheless, uncertainties during mission execution considerably increase the risk of UAV malfunctions [5]. These uncertainties impose stringent demands on reliability and stability, especially in safety-critical applications requiring advanced control systems and robust fault detection. Compared to other UAV types, FW-UAVs possess more complex configurations and typically serve specialized, demanding tasks. Their intricate subsystems may adversely affect critical performance metrics [6] and can lead to severe property damage and safety hazards when faults occur. Therefore, flight-data-based fault diagnosis for FW-UAVs holds considerable practical significance.

Fault diagnosis technology is essential for improving system stability and reducing failure rates in UAVs. A comprehensive review of existing methods categorizes them into four groups:

1. **Hardware redundancy–based methods.** These approaches improve UAV reliability by integrating redundant components, thereby substantially enhancing system robustness [7]. However, redundancy increases costs, system complexity, and

* Corresponding author.
Email addresses: gs.yyi24@gzu.edu.cn (R. Yi), jglu@gzu.edu.cn (J. Lu), xhtang@gzu.edu.cn (X. Tang).

https://doi.org/10.1016/j.asoc.2026.114589
Received 14 August 2025; Received in revised form 1 December 2025; Accepted 5 January 2026
Available online 7 January 2026
1568-4946/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

hardware-fault risk. Moreover, added weight and volume reduce payload capacity, limiting practical applications.

2. **Expert knowledge-based methods.** These methods provide enhanced interpretability and lower false alarm rates relative to hardware-based techniques. Some implementations apply hidden Markov models (HMMs) [8] and adaptive neuro-fuzzy inference systems (ANFIS) [9] to diagnose navigation sensor faults. However, reliance on subjective and limited expert knowledge restricts scalability and generalization.
3. **Model-driven methods.** Valued for computational efficiency and reliability, these techniques depend on precise mathematical models. For example, a differential flatness-based approach detects sensor and actuator faults by comparing analytical and actual measurements to compute residuals [10]. Another framework uses an improved Kalman filter to update neural network parameters in real time [11]. Despite these advantages, modeling inaccuracies often result in increased false positives.
4. **Data-driven methods.** These approaches have become prominent for learning from historical flight data without explicit models. Their adaptability and minimal expert-knowledge requirement render them suitable for complex nonlinear systems. One method extracts spatiotemporal features via neural networks and employs Long Short-Term Memory (LSTM) networks for anomaly recovery [12]. Another integrates discrete wavelet transforms with deep neural networks for time–frequency fault diagnosis [13]. Although data-driven approaches dominate UAV fault diagnosis research, they rely heavily on large-scale fault data, which are rarely available in real-world UAV operations.

In real-world UAV operations, UAV fault data are difficult to obtain owing to the uniqueness of missions and the operational risks and safety constraints involved. To address this challenge, existing approaches frequently generate synthetic fault datasets via simulation and utilize techniques such as time–frequency feature extraction and domain adaptation to improve detection performance [14,15]. Additionally, some studies acquire real fault data by injecting faults during flight tests or simulating specific fault conditions to obtain representative training data [16,17]. However, simulated data do not capture actual dynamic behaviors and noise characteristics. Conversely, real-time fault injection incurs substantial costs and operational risks, especially in complex environments where UAVs maintain stable control under varying conditions. Consequently, data-driven UAV fault diagnosis faces a critical small-sample learning challenge. To address this issue, various deep-learning methods have been explored. The Siamese Hybrid Neural Network (SHNN) combines one-dimensional convolutional neural networks (1D-CNN) for spatial feature extraction with LSTM networks for temporal modeling, utilizing distance-based similarity measures for classification [18]. To further enhance small-sample performance, multi-scale CNN architectures integrating adaptive learning mechanisms, such as Kalman filter–enhanced optimization [19], and time–frequency feature–based encoder–decoder networks [20] have been proposed. Moreover, recent studies have incorporated attention mechanisms to enhance discriminative feature learning. The adaptive Siamese Network (ASN) employs channel-wise attention modules that dynamically recalibrate multi-channel feature responses to reinforce spatial–temporal coupling [21]. Similarly, the heterogeneous deep multitask learning (HDMTL) framework combines CNN and LSTM modules with adaptive attention–based feature fusion [22].

Recently, graph neural networks (GNNs) have emerged as a robust paradigm for modeling complex relational structures in multivariate systems; in UAV-related applications, graph-based architectures have proven effective at capturing inter-component dependencies through adaptive topology learning. Several GNN-based approaches have been developed specifically for UAV fault diagnosis. DIGPGNN [23] constructs a feature-similarity–based directed graph and employs adaptive bidirectional aggregation to enhance homophilic neighborhood representations, moreover introducing an efficient graph transformer with global self-attention to connect distant homophilic nodes. CLGNN [24] models sensor-relation graphs with causal interventions during feature extraction to suppress non-causal components and improve interpretability. He et al. [25] constructed cross-component sensor-relation graphs and combined spatial graph attention with GRU networks to capture temporal dynamics. Li et al. [26] introduced MFFTD, which leverages multi-scale convolution and LSTM on temporal dynamic graphs to capture spatio-temporal correlations and employs mutual information-based multi-head feature fusion to address heterogeneity imbalance. Despite these advances, most methods depend on predefined static graph structures or separate temporal modules, failing to jointly learn adaptive dynamic graphs and multi-resolution temporal patterns within a unified framework. This limitation becomes critical in small-sample scenarios wherein structural adaptability and hierarchical temporal modeling are essential for robust fault detection.

UAV flight data exhibit complex spatiotemporal dependencies and dynamic multivariate structures that challenge effective modeling. Although these methods demonstrate notable progress, they often treat spatial structural learning separately from temporal feature extraction or rely on fixed topological assumptions that may not generalize across varying operational conditions. Specifically, static graph structures fail to adapt when UAV sensor relationships evolve with flight conditions, limiting their generalization capability. Meanwhile, single-scale attention mechanisms focus on uniform temporal granularity, missing either fine-grained transient faults or long-term dynamic drifts. Furthermore, simple feature fusion strategies ignore the structural heterogeneity between graph-based spatial and sequence-based temporal features, introducing noise and redundancy. Consequently, their ability to capture the coupled evolution of spatial dependencies and temporal dynamics remains limited, especially in data-scarce environments. These observations underscore the necessity for data-efficient models with adaptive spatiotemporal representations.

To overcome these challenges, we propose a novel framework—**D**ynamic **G**raph-enhanced **M**ulti-scale **A**ttention **N**etwork (DGMANet)—for robust fault diagnosis in FW-UAVs under small-sample conditions. DGMANet integrates three modules to capture local and global dynamics. First, a dynamic graph construction module adaptively models localized structural dependencies and inter-feature relationships. Second, a multi-scale attention mechanism (MSAM) extracts hierarchical temporal features at multiple resolutions, enabling the network to concentrate on salient time-dependent patterns. Third, an adaptive feature aggregation module (AFAM) fuses local and global representations, enhancing complementarity while suppressing noise and redundancy. Together, these components form a cohesive architecture capable of accurately identifying faults in complex, data-scarce operational environments. The primary contributions of this work are as follows:

- We introduce a dynamic graph construction mechanism that models localized topological dependencies and adaptive inter-feature correlations, thereby enabling structural representation learning without reliance on large-scale data.
- We design a multi-scale attention module that extracts hierarchical temporal features and selectively emphasizes salient patterns across varying time resolutions, thus improving the capacity of the model for capturing complex temporal dynamics.
- We implement an adaptive feature aggregation strategy to merge local and global representations, improving feature complementarity while effectively mitigating redundancy and noise in spatiotemporal signals.
- Extensive validation on four real-world FW-UAV flight datasets demonstrates that DGMANet achieves high diagnostic accuracy.

The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 presents the problem formulation and details

the implementation of the proposed DGMANet; Section 4 describes the experimental setup, baseline methods, analysis, ablation studies, and visualization results; and Section 5 concludes the paper and outlines potential directions for future research.

2. Related works

In this section, we provide an overview of the theoretical foundations and recent advances in small-sample UAV fault diagnosis, dynamic graph neural networks, and local and global spatiotemporal modeling.

2.1. Small-sample UAV fault diagnosis

The rapid proliferation of UAVs in complex mission environments has rendered fault diagnosis essential for ensuring system safety, autonomy, and robustness. To address limited-sample scenarios, researchers proposed an adaptive Siamese network with channel attention that identifies fault patterns from multi-channel sensor data with few-shot adaptability [21]. For real-time onboard recognition, a lightweight CNN–LSTM architecture was developed and optimized for embedded platforms, enabling efficient fault cause detection directly from raw sensor signals [27]. A meta-learning framework with adaptive learning rates was introduced to enhance generalization and enable rapid adaptation to new fault types with few labeled samples [28]. To improve anomaly detection under heterogeneous data conditions, a hybrid multi-modal neural network was introduced that leverages attention mechanisms and soft-voting strategies for robust cross-domain fault identification [29]. Furthermore, a knowledge-complementary framework was proposed to jointly exploit temporal and spatial features in identifying fault locations and severity, thereby enhancing diagnostic accuracy and generalization [22]. Few-shot fault diagnosis substantially improves UAV safety and robustness in data-scarce, complex operational scenarios. Nevertheless, most existing methods rely on episodic training and support-query matching paradigms, which limit their applicability when representative samples are scarce under diverse flight conditions.

2.2. Dynamic graph neural networks

GNNs have exhibited strong capabilities in processing non-Euclidean data and modeling spatial dependencies. Early contributions introduced Graph Convolutional Networks (GCNs), which update node features by aggregating information from adjacent and self-nodes [30]. GCNs are broadly categorized into spectral [31] and spatial approaches. Spectral methods perform convolution in the graph spectral domain, whereas spatial methods extract structural features through convolution over local neighborhoods. Recent studies have leveraged GCNs for various fault diagnosis applications. Some integrate GCNs with structure analysis (SA) to convert pre-diagnostic results into association graphs for fault identification [32]. Others incorporate temporal modeling via time recursive GNNs that embed recurrent neural networks (RNNs) for dynamic system monitoring [33]. Multi-scale architectures, such as MS-GCN [34] and MST-BGCT [35], refine the extraction of subtle spatiotemporal features by combining GCNs with iterative modules or Transformer-based encoders. Despite these advancements, the application of GCNs to small-sample fault diagnosis in FW-UAVs remains largely unexplored. Given their robust spatial modeling capabilities, GCNs hold substantial potential for capturing local topological structures and spatial pairwise dependencies in time series data, offering avenues to advance fault diagnosis performance in this challenging domain. Therefore, further research is needed to explore how temporal dynamic graphs can be applied from a global perspective to address small-sample flight fault diagnosis.

2.3. Local and global spatiotemporal modeling

Recent studies have indicated that integrating local features with global correlations substantially enhances temporal feature extraction in time series modeling [36]. FW-UAV flight data comprise multiple time-varying variables and exemplify multivariate time series data. To capture spatiotemporal dynamics from raw sensor inputs, models commonly employ combinations of convolutional neural networks (CNNs) and bidirectional or unidirectional long short-term memory (BiLSTM/LSTM) networks, in which CNNs extract spatial features and LSTM variants capture local temporal dependencies [18,37]. Alternative to sequence-based models, some approaches transform one-dimensional time series into two-dimensional tensors to enable CNNs to capture both local and global temporal patterns simultaneously [38]. The multi-head attention architecture offers advantages in modeling long-range dependencies within time series data, making it well suited to address the complex temporal dynamics inherent in FW-UAV flight data [39]. Recent methods such as the multiscale periodic time series network (MPTSNet) integrate convolution and attention modules within a multi-scale framework to jointly model local patterns and global correlations [40]. However, LSTM-based models remain limited in capturing global temporal structures, and either strictly serial or parallel processing of local and global features can restrict their interaction, potentially resulting in the loss of critical contextual information. These limitations underscore the need for more adaptive, integrated architectures that fully exploit the spatiotemporal characteristics of FW-UAV flight data. However, single-scale attention mechanisms and simple fusion strategies fail to capture multi-granularity temporal patterns while overlooking the structural heterogeneity between spatial and temporal features. In this study, we propose a MSAM module for hierarchical temporal modeling and an AFAM module for heterogeneous spatiotemporal fusion.

3. Methodology

In this section, we first provide a systematic definition of the symbols used in the FW-UAV fault diagnosis problem. Subsequently, we present a comprehensive description of the DGMANet architecture, including the data embedding module, the dynamic graph-enhanced module (DGM), the global dependencies extractor (GDE), the MSAM module, and the AFAM module. Subsequently, the pooling and fully connected layers used for classification are detailed. A rigorous time-complexity analysis of DGMANet is also provided.

3.1. Model architecture overview

The overall DGMANet framework is illustrated in Fig. 1. Initially, DGMANet preprocesses the input time series data. Subsequently, a dual-branch architecture is adopted for feature modeling: one branch employs DGM to extract spatial-local topological structures and dynamic correlations among feature variables, while the other branch leverages GDE to capture long-range global dependencies and prominent temporal patterns. The GDE branch output is further processed by MSAM, which characterizes multi-scale temporal dynamics while maintaining computational efficiency. The features from the two branches are then adaptively fused through AFAM, enhancing feature complementarity and reducing noise interference. Finally, the fused features pass through a dropout layer and an activation function, followed by a fully connected layer to generate the fault classification results. The model is described in detail in the following subsections.

3.2. Problem formulation

Definition 1. (Flight data). UAV flight data are acquired from onboard sensors (e.g., accelerometers and magnetometers) and transmitted to the ground control system (GCS), providing essential measurements such as position, acceleration, and pitch angle. These data serve as inputs for fault diagnosis. We represent the flight data as a multivariate time series

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,T} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,T} \\ \vdots & \vdots & \ddots & \vdots \\ x_{C,1} & x_{C,2} & \cdots & x_{C,T} \end{bmatrix} \in \mathbb{R}^{C \times T}, \tag{1}$$

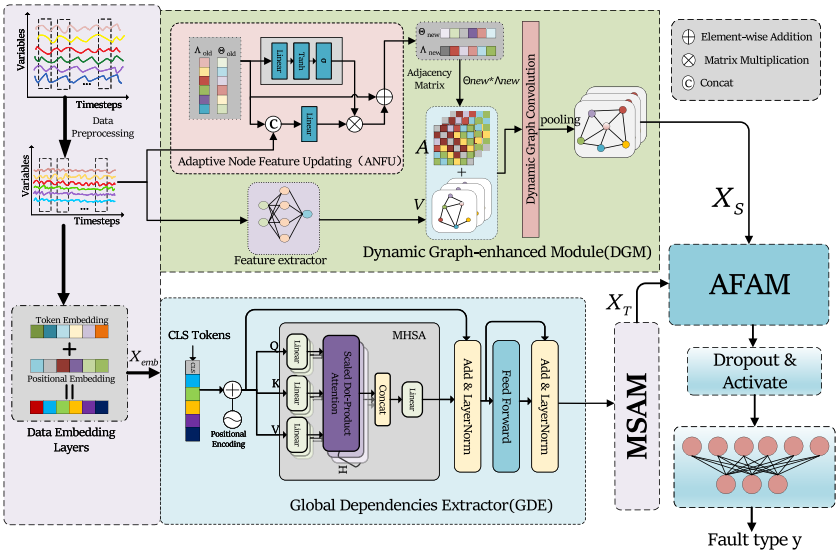


Fig. 1. The framework of DGMANet consists of three stages: data preprocessing, dual-branch feature extraction, and fault diagnosis. Preprocessed flight data $X \in \mathbb{R}^{B\times T\times C}$ is fed into the temporal branch, where the data embedding layer generates $X_{\text{emb}} \in \mathbb{R}^{B\times T\times d}$, followed by GDE and MSAM to extract temporal features $X_T \in \mathbb{R}^{B\times T\times d}$. In the spatial branch, the feature extractor and ANFU generate adjacency matrix $A \in \mathbb{R}^{N\times N}$, and dynamic graph convolution transforms node features $V \in \mathbb{R}^{N\times d}$ across hidden layers, yielding spatial features $X_S \in \mathbb{R}^{B\times d'}$ after pooling. AFAM fuses X_T and X_S via element-wise multiplication $\otimes$ and addition $\oplus$ to produce the final output. Finally, fully connected layers output fault classification $y \in \mathbb{R}^{B\times K}$, where B, T, C, N, d, and K represent batch size, sequence length, input channels, number of nodes, hidden dimension, and number of fault categories, respectively.

where the t-th column $x_t \in \mathbb{R}^C$ represents the observation vector at time t. For training, the data are organized into mini-batches $\tilde{X} \in \mathbb{R}^{B\times C\times T}$ with corresponding labels $y \in \mathbb{R}^B$. Here, B denotes the batch size, C indicates the number of channels, and T represents the feature length.

Definition 2. (Graph structure). A dynamic graph can be defined as $G = (V, E, A, X)$, where $V = \{v_1, v_2, \ldots, v_C\}$ denotes the set of C nodes corresponding to feature variables, and $E \subseteq V \times V$ represents the set of edges that capture pairwise relationships. The adjacency matrix $A = [a_{ij}] \in \mathbb{R}^{C\times C}$ encodes the latent topological dependencies among nodes, where $a_{ij} \neq 0$ if an edge $e_{ij} \in E$ exists between v_i and v_j. The node feature matrix $X \in \mathbb{R}^{C\times T}$ collects the temporal feature vectors for all nodes in the graph.

Definition 3. (Fault diagnosis tasks). The UAV fault diagnosis problem is formulated as a supervised multivariate time series classification task. The dataset consists of B samples $\tilde{X} = \{X_1, X_2, \ldots, X_B\}$, where each sample $X_i \in \mathbb{R}^{C\times T}$ contains C sensor features across T time steps. The corresponding labels are $y = \{y_1, y_2, \ldots, y_B\}$, with

$$y_i \in \begin{cases} \{0,1\}, & \text{binary classification (normal vs. faulty)}, \\ \{1,2,\ldots,c\}, & \text{multi-class classification with } c \text{ fault types}, \end{cases} \tag{2}$$

where c denotes the total number of fault categories. The objective is to learn a discriminative mapping function

$$f : \mathbb{R}^{C\times T} \to \mathcal{Y}, \quad \mathcal{Y} = \begin{cases} \{0,1\}, & \text{binary}, \\ \{1,2,\ldots,c\}, & \text{multi-class}, \end{cases}$$

that accurately classifies each input sample into its corresponding fault category.

3.3. Data embedding module

Effective representation of multivariate time series is essential for enhancing downstream fault diagnosis performance. Accordingly, the proposed data embedding module projects the raw input $\tilde{X} \in \mathbb{R}^{B\times C\times T}$ into a high-dimensional latent space, incorporating both sensor features and temporal positional information.

We employ a dual-branch embedding strategy comprising token and positional embeddings.

- **Token embedding**: A one-dimensional convolution is applied across the channel dimension to project the multivariate input into a feature space of dimension d_{embed}, yielding:

$$E_{\text{token}} = \text{Conv1d}(\tilde{X}), \quad E_{\text{token}} \in \mathbb{R}^{B\times T\times d_{\text{embed}}}. \tag{3}$$

- **Positional embedding**: To preserve temporal order, we generate position-aware vectors via sine and cosine functions at each time step:

$$E_{\text{pos}}(t,i) = \begin{cases} \sin\left(t/10000^{2i/d_{\text{embed}}}\right), & \text{if } i \text{ is even}, \\ \cos\left(t/10000^{2i/d_{\text{embed}}}\right), & \text{if } i \text{ is odd}, \end{cases} \tag{4}$$

where t indicates the time index and i the embedding-dimension index.

We derive the final embedding by element-wise addition $\oplus$ of the token and positional embeddings and apply dropout regularization:

$$X_{\text{embed}} = \text{Dropout}(E_{\text{token}} \oplus E_{\text{pos}}), \tag{5}$$

where $\oplus$ denotes element-wise addition. This representation encodes both spatial (feature-wise) and temporal information and serves as input to subsequent modules.

3.4. DGM

Multivariate UAV time-series data often exhibit complex non-Euclidean and dynamically evolving spatial dependencies with flight conditions, posing significant challenges for conventional CNNs that operate on fixed receptive fields and regular grids. Furthermore, traditional static graph structures assume fixed inter-sensor relationships, failing to capture such evolving topologies and limiting generalization under diverse operational scenarios. To overcome these limitations, we propose a dynamic graph-enhanced module (DGM) that learns adaptive graph

structures and performs hierarchical message passing to effectively model latent spatial correlations among fault-related variables.

Given the input tensor $X \in \mathbb{R}^{B\times C\times T}$, with B as the batch size, C as the number of fault variables, and T as the temporal length, we initialize two learnable node-embedding matrices: a source embedding $\Lambda \in \mathbb{R}^{C\times d}$ and a target embedding $\mathcal{E} \in \mathbb{R}^{d\times C}$. To integrate temporal dynamics into the graph construction process, we employ a gated fusion mechanism defined as follows:

$$G_1 = \mathrm{ReLU}\left(\tanh(W_1[X,\Lambda])\right), \quad G_2 = \mathrm{ReLU}\left(\tanh(W_2[X,\mathcal{E}^\top])\right), \tag{6}$$

$$\Lambda' = \Lambda + G_1 \odot \phi(X), \quad \mathcal{E}' = \mathcal{E} + G_2 \odot \psi(X) \tag{7}$$

where $[\cdot,\cdot]$ denotes channel-wise concatenation, ϕ and ψ represent linear transformations, and $\odot$ signifies element-wise multiplication. These updated embeddings combine static structural priors with input-aware temporal context.

To construct a data-adaptive graph structure, we compute pairwise node similarity using the updated source and target embeddings, producing a raw adjacency matrix A with non-negative edge weights enforced by a ReLU activation:

$$\mathbf{A} = \mathrm{ReLU}(\Lambda'\mathcal{E}'). \tag{8}$$

To stabilize propagation and mitigate high-degree node influence, we add self-loops and apply symmetric normalization:

$$\hat{A} = D^{-1/2}(A+I)D^{-1/2}, \tag{9}$$

where $D_{ii} = \sum_j (A+I)_{ij}$ defines the degree matrix. This operation ensures numerical stability and balanced information aggregation across the graph.

Using the normalized adjacency matrix $\hat{A}$, we perform multi-order graph convolution to capture both local and higher-order spatial dependencies. At each propagation order k, node features are iteratively updated via

$$X^{(k)} = \mathrm{ReLU}\left(\mathrm{BN}\left(\mathrm{Conv1D}(\hat{A}X^{(k-1)}W)\right)\right), \quad k = 1,2,\ldots,K. \tag{10}$$

The representations across all K orders are concatenated and processed by a multilayer perceptron (MLP) to yield refined hierarchical features:

$$H_1 = \mathrm{ReLU}\left(\mathrm{MLP}([X^{(0)},\ldots,X^{(K)}])\right),$$
$$H_2 = \mathrm{ReLU}\left(\mathrm{MLP}([H_1^{(0)},\ldots,H_1^{(K)}])\right). \tag{11}$$

To enhance generalization while preserving input semantics, we incorporate a residual connection from the original input X to the transformed output H_2, followed by dropout and adaptive average pooling:

$$Y = \mathrm{AvgPool}(\mathrm{Dropout}(H_2) + X). \tag{12}$$

This architecture enables the model to exploit both immediate and long-range topological dependencies, maintain training stability, and mitigate the over-smoothing commonly observed in deep GCNs.

3.5. GDE

UAV multivariate time-series data exhibit hierarchical temporal structures with fault-relevant patterns spanning multiple resolutions, from local fluctuations to global trends. Conventional sequential models struggle to capture such long-range dependencies efficiently, often losing critical contextual information across distant time steps. Therefore, effective modeling of hierarchical global dependencies in multivariate UAV time-series data is crucial for capturing the temporal patterns across multiple resolutions. To address this challenge, we propose a global dependencies extractor (GDE) employing a multi-head attention mechanism to enhance the extraction of long-range contextual relationships and time-dependent fault signatures.

The input flight data is processed by an embedding layer, yielding a tensor $X_{\mathrm{embed}} \in \mathbb{R}^{B\times d\times T}$, where B denotes the batch size, d the embedding dimension, and T the timestep length. We then permute the tensor to treat each timestep as an individual token, resulting in $X' \in \mathbb{R}^{B\times T\times d}$.

Inspired by BERT [41] and MPTSNet [40], we include a learnable classification token, $Z_{\mathrm{cls}} \in \mathbb{R}^{1\times 1\times d}$, which is prepended to the input sequence prior to GDE processing. This token aggregates global representative information from all tokens in each layer and, owing to its lack of explicit semantic content, permits balanced integration of inter-token semantics across the sequence. Consequently, we concatenate Z_{cls} to the beginning of the token sequence X' to obtain the augmented input as follows:

$$X_{\mathrm{aug}} = [Z_{\mathrm{cls}} \,||\, X'] \in \mathbb{R}^{B\times(T+1)\times d} \tag{13}$$

Temporal ordering information is integrated into the input sequence by applying a fixed sinusoidal positional encoding, consistent with the original multi-head attention design. For each position pos $\in \{0,1,\ldots,T\}$ and embedding dimension $i \in \{0,1,\ldots,d-1\}$, the positional encoding $\mathrm{PE}(\mathrm{pos},i)$ is defined as align (4). After applying positional encoding, the augmented input sequence X_{aug} produces the initial feature representation H. Subsequently, multi-head attention captures global dependencies across temporal positions. Specifically, each temporal feature segment H^i is transformed into query, key, and value representations as follows:

$$Q^i = W_Q H^i, \quad K^i = W_K H^i, \quad V^i = W_V H^i \tag{14}$$

where W_Q, W_K, and W_V represent learnable weight matrices. The multi-head attention is then computed as follows:

$$\mathrm{head}_i = \mathrm{Attention}(Q^i,K^i,V^i) = \mathrm{Softmax}\left(\frac{Q^i(K^i)^\top}{\sqrt{d_k}}\right)V^i \tag{15}$$

$$\mathrm{MHSA} = \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_h)W^O \tag{16}$$

where h denotes the number of attention heads, d_k represents the dimension of each head, and W^O is a learnable output projection matrix. This mechanism enables the model to capture complex dependencies across all time series segments. Thereafter, the sequence passes through L stacked attention encoder layers. Each layer incorporates multi-head self-attention (MHSA), a feed-forward network (FFN), residual connections, and layer normalization as follows:

$$H'_l = \mathrm{LayerNorm}(H_{l-1} + \mathrm{MHSA}(H_{l-1})),$$
$$H_l = \mathrm{LayerNorm}(H'_l + \mathrm{FFN}(H'_l)) \tag{17}$$

where $l = 1,\ldots,L$. The output of the final encoder layer is $H^{(L)} \in \mathbb{R}^{(T+1)\times B\times d}$.

Following the final GDE layer, the output tensor $H^{(L)} \in \mathbb{R}^{(T+1)\times B\times d}$ splits into a global classification token and a set of temporal sequence tokens. The first token yields the CLS token $X_{\mathrm{CLS}} = H^{(L)}[0] \in \mathbb{R}^{B\times d}$, whereas the remaining tokens form the sequence representation $X_{\mathrm{seq}} = H^{(L)}[1:] \in \mathbb{R}^{T\times B\times d}$. X_{CLS} is expanded along the temporal axis to match X_{seq}, and they are concatenated along the last dimension to produce the fused representation $X_{\mathrm{fused}} \in \mathbb{R}^{T\times B\times 2d}$. The fused sequence is projected into the original feature space via a linear transformation:

$$Y = X_{\mathrm{fused}}W + b \in \mathbb{R}^{T\times B\times d} \tag{18}$$

where $W \in \mathbb{R}^{2d\times d}$ is a learnable weight matrix and $b \in \mathbb{R}^d$ is the corresponding bias term. The output is transposed to $Y_{\mathrm{out}} \in \mathbb{R}^{B\times d\times T}$ to align with the original input format. This representation integrates global contextual information with fine-grained temporal dynamics, yielding a unified, semantically enriched feature tensor ideal for fault diagnosis tasks.

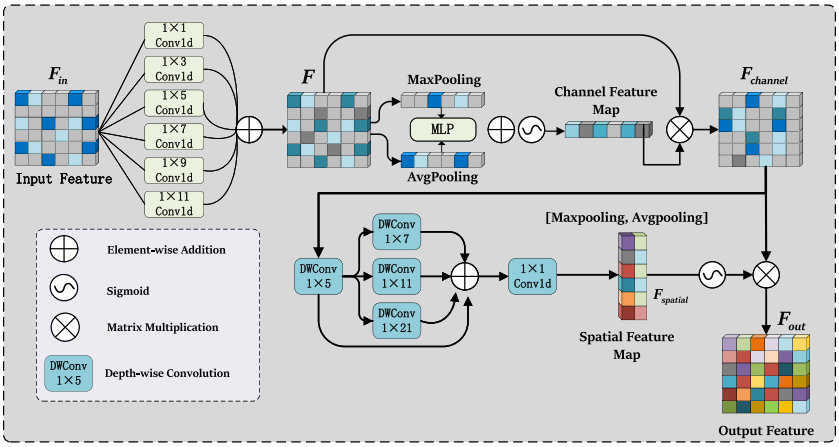


Fig. 2. Structure diagram of MSAM. The input feature F_{in} is fused by a multi-scale convolutional extractor to obtain F. Subsequently, F sequentially passes through channel attention and spatial attention mechanisms. The channel attention generates $F_{channel}$ through pooling operations and an MLP, while the spatial attention generates $F_{spatial}$ through multi-scale depth-wise convolutions. Finally, the output feature F_{out} is produced through adaptive fusion.

3.6. MSAM

UAV flight sequences frequently exhibit complex dynamic behaviors with temporal dependencies spanning multiple time scales. Single-scale attention mechanisms often fail to simultaneously capture both rapid transient anomalies and slow-evolving degradation patterns, limiting their effectiveness in small-sample fault diagnosis scenarios. As a consequence, capturing temporal dependencies across multiple time scales remains crucial for modeling UAV flight sequences, which frequently exhibit complex dynamic behaviors. To address this challenge, we introduce MSAM, a multi-scale attention module that integrates parallel temporal convolutions with channel-wise attention mechanisms. This design enhances the capacity of the model to extract salient temporal patterns while mitigating redundant or noisy activations, thereby improving the robustness and discriminability of sequence representations.

The multi-scale extractor within MSAM employs parallel one-dimensional convolutional branches with varying kernel sizes, which enable the model to capture both short-term and long-term dynamics present in the input sequences. The overall architecture of the MSAM module is depicted in Fig. 2.

Let the input feature be denoted as $X \in \mathbb{R}^{B\times T\times C}$, where B, T, and C represent the batch size, temporal dimension, and number of channels, respectively. The module employs N convolutional branches with kernel sizes $k_i = 2i + 1$ for $i = 0, 1, \ldots, N-1$, and zero-padding is applied to preserve temporal length. The multi-scale convolutional output is computed as

$$F_{multi} = \frac{1}{N}\sum_{i=1}^{N} \mathrm{Conv}_{k_i}(X) \tag{19}$$

This parallel structure enables temporal feature extraction across multiple receptive fields, allowing the model to capture patterns at various frequencies.

To further refine these features, the module incorporates a temporal attention mechanism with two components: Channel Attention (CA) and Spatial Attention (SA). The CA module emphasizes informative channels, whereas the SA module selects critical time steps.

Initially, a pointwise 1×1 convolution is applied to the multi-scale features, followed by GELU activation to enhance inter-channel interactions as follows:

$$F' = \mathrm{GELU}(\mathrm{Conv}_{1\times 1}(F_{multi})) \tag{20}$$

Then, global average pooling and max pooling are performed, and their outputs pass through a shared multilayer perceptron (MLP) to compute channel-wise attention weights. The enhanced output after channel attention is given by Eq. (22).

$$M_c = \sigma\left(\mathrm{MLP}(\mathrm{AvgPool}(F')) + \mathrm{MLP}(\mathrm{MaxPool}(F'))\right) \tag{21}$$

$$\mathrm{CA}(F') = F' \otimes M_c \tag{22}$$

Subsequently, to highlight salient temporal positions, the channel-attended feature is processed by depthwise separable convolutions with various kernel sizes, including a preceding 5×5 Branch_0. The resulting feature map is then pooled and fused via a 1×1 convolution, followed by a sigmoid activation to generate spatial attention weights M'_s. Finally, the output M'_s, refined by both channel and spatial attention, is obtained as in Eq. (25).

$$M_s = \sum_{i=0}^{M} \mathrm{DwConv}_{k_i}(\mathrm{Branch}_i(\mathrm{CA}(F'))) \tag{23}$$

$$M'_s = \sigma\left(\mathrm{Conv}_{1\times 1}\left[\mathrm{AvgPool}(M_s)\|\mathrm{MaxPool}(M_s)\right]\right) \tag{24}$$

$$\mathrm{SA}(F') = \mathrm{CA}(F') \otimes M'_s \tag{25}$$

where $\mathrm{CA}(\cdot)$ denotes the channel attention mechanism, $\mathrm{DwConv}_{k_i}(\cdot)$ indicates the depthwise separable convolution with kernel size k_i, $\|$ indicates feature concatenation, $\sigma(\cdot)$ indicates the sigmoid function, and $\otimes$ represents element-wise multiplication.

By integrating multiscale temporal encoding with adaptive attention, the MSAM model effectively captures temporal structures while enhancing robustness against irrelevant or noisy information. This approach is particularly beneficial for UAV time-series anomaly detection, where salient dynamic patterns vary in scale and intensity.

3.7. AFAM

Although GDE architectures excel at capturing global temporal dependencies and DGM models effectively represent spatial local topologies, directly combining their outputs often causes modality misalignment due to structural heterogeneity between sequence-based and graph-based representations. To fuse local and global representations, enhance complementarity, and reduce noise and redundancy, we design AFAM to enable deep interactions across modalities, thus bridging global sequence modeling and localized spatial representations. AFAM comprises two main components: a cross-channel attention mechanism that captures inter-modal channel dependencies and a spatial attention mechanism that enhances localized features. This hierarchical design

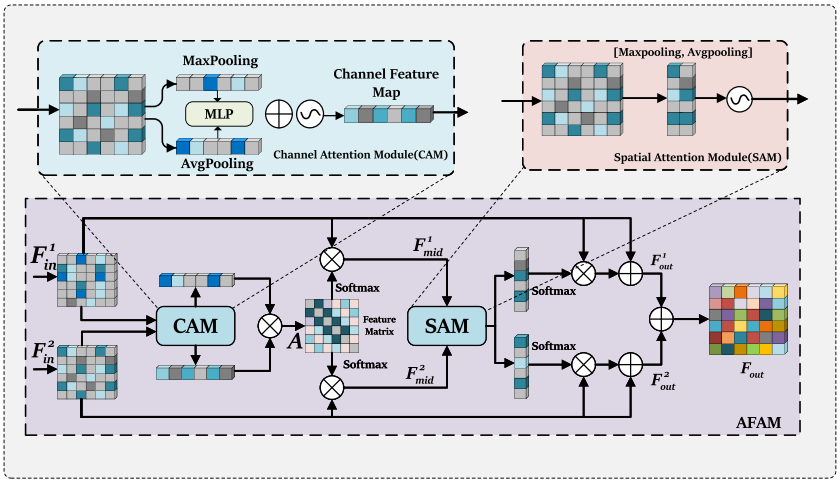


Fig. 3. The framework of AFAM. The CAM processes the dual input features $\mathbf{F}_{in}^{1}$ and $\mathbf{F}_{in}^{2}$ to generate a feature matrix $\mathbf{A}$, which is weighted with the input features to produce intermediate representations $\mathbf{F}_{mid}^{1}$ and $\mathbf{F}_{mid}^{2}$. The SAM subsequently refines the intermediate representations to generate $\mathbf{F}_{out}^{1}$ and $\mathbf{F}_{out}^{2}$, which are fused to produce the final output $\mathbf{F}_{out}$.

supports the fusion of complementary features across the GDE and DGM branches. The overall architecture of AFAM is depicted in Fig. 3.

To formally describe the operations within AFAM, we first define the input features extracted from the GDE encoder and the DGM branch. Let $X_i \in \mathbb{R}^{B\times T\times C}$ $(i = 1, 2)$ denote the input features from the two branches, where B represents the batch size, T the sequence length, and C the number of channels. AFAM first employs a lightweight cross-channel attention mechanism to capture inter-modal channel dependencies. Global descriptors are obtained via average and max pooling along the temporal dimension and subsequently processed through a shared multilayer perceptron (MLP) followed by a sigmoid activation.

$$M_i = \sigma\left(\mathrm{MLP}(\mathrm{AvgPool}(X_i)) + \mathrm{MLP}(\mathrm{MaxPool}(X_i))\right), \quad i = 1, 2 \tag{26}$$

Here, MLP designates the shared fully connected transformation, and $\sigma(\cdot)$ denotes the sigmoid activation. The resulting vectors computed a cross-channel attention matrix that encapsulates interdependencies among feature channels across modalities and permits bidirectional information exchange.

$$M_{\text{cross}} = \mathrm{Softmax}(M_1 \cdot M_2^{\top}) \tag{27}$$

$$X_1^{\text{att}} = M_{\text{cross}} \cdot X_1, \quad X_2^{\text{att}} = M_{\text{cross}}^{\top} \cdot X_2 \tag{28}$$

To emphasize the salient temporal information, the module implements spatial attention by pooling average and max features along the channel dimension and concatenating their outcomes along the temporal axis.

$$S_i = \mathrm{Concat}(\mathrm{AvgPool}(X_i^{\text{att}}), \mathrm{MaxPool}(X_i^{\text{att}})), \quad i = 1, 2 \tag{29}$$

Subsequently, two shared 1×1 convolutional layers with ReLU activation compute the temporal attention weights, with learnable weights:

$$A_{ti} = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot S_i)), \quad i = 1, 2 \tag{30}$$

where W_1 and W_2 enhance the capacity of the model to selectively focus on temporally informative sequence regions. To preserve integrity and ensure stable gradient flow, the attention-enhanced features are integrated with their inputs via residual connections:

$$X_i^{\text{out}} = X_i^{\text{att}} \odot A_{ti} + X_i, \quad i = 1, 2 \tag{31}$$

The final AFAM output is computed by aggregating the enhanced feature representations:

$$X_{\text{final}} = X_1^{\text{out}} + X_2^{\text{out}} \in \mathbb{R}^{B\times C\times T} \tag{32}$$

Notably, the proposed module supports efficient and adaptive modality interaction while maintaining a compact computational footprint, rendering it suitable for large-scale or resource-constrained applications. The model was optimized using standard cross-entropy loss.

$$\mathcal{L}_{loss} = -\frac{1}{N}\sum_{k=1}^{N} y_k \log \hat{y}_k \tag{33}$$

AFAM effectively fuses temporal and spatial representations through hierarchical attention-based interactions, thus enhancing the expressiveness and semantic coherence of the fused features.

The entire DGMANet framework is summarized in Algorithm 1.

3.8. Time complexity analysis

In this subsection, we analyze the framework's time complexity according to its data flow. The architecture comprises four primary modules: DGM, GDE, MSAM, and AFAM. Given an input feature map $X \in \mathbb{R}^{B\times C\times T}$, with B denoting batch size, C channel count (or embedding dimension), and T temporal sequence length, each module's time complexity is evaluated as follows.

First, the DGM module dynamically captures topological relationships among graph nodes by computing a dynamic adjacency matrix through pairwise operations over node embeddings, incurring $O(BN^2)$. Multi-order GCN layers then propagate features across the graph, adding $O(BN^2)$ from dense matrix multiplications. Gating and linear transformations incur minor overhead, so the dominant complexity of DGM remains $O(BN^2)$.

Thereafter, GDE models long-range temporal dependencies across the sequence. Its multi-head attention mechanism performs pairwise interactions over sequence elements, resulting in $O(BT^2D)$ per layer. Each FFN layer incurs $O(BTDF)$, where F represents the FFN hidden dimension. With *num_layers* stacked GDE blocks, the overall time complexity of this module is $O(\text{num_layers} \times (BT^2D + BTD^2 + BTDF))$.

Subsequently, the MSAM module enhances temporal modeling via multi-scale convolutions and attention mechanisms. In particular, multiple 1D convolution branches with distinct kernel sizes extract multi-scale features, yielding a complexity of $O(BC^2\,T)$. Additional components—channel attention and spatial attention—contribute minor terms that do not alter the overall complexity order. Thus, the primary time complexity of this module remains $O(BC^2\,T)$.

Finally, the AFAM module integrates two input sequences using dual attention mechanisms. The channel-wise cross-attention computation requires $O(BC^3)$ operations due to batch-wise matrix multiplications. Applying the attention maps to the sequences requires $O(BC^2\,T)$, where

Algorithm 1 The algorithmic framework of DGMANet.

Input: The flight data $\widetilde{X} = \{X_1, \ldots, X_B\} \in \mathbb{R}^{B\times C\times T}$
Output: The corresponding predicted fault categories $Y = \{y_1, \ldots, y_B\} \in \mathbb{R}^{1\times B}$, Accuracy, Macro-F1, AUROC
Randomly sample training, validation and testing sets from $\widetilde{X}, Y$
Initialize optimizer AdamW and learning rate scheduler ReduceLROnPlateau
for $epoch = 1$ to $Epochs$ **do**
  $\widetilde{X}_G$ = DataEmbedding($\widetilde{X}$) // Data Embedding using Eqs. (3)–(5)
  for batch (X_{batch}, y_{batch}) in training set **do**
    $\hat{A} \leftarrow \{\widetilde{X}\}$ // Generating the graph adjacency matrix using Eqs. (6)–(9)
    X_L = DGM($\widetilde{X}, \hat{A}$) // Dynamic graph convolution using Eqs. (10)–(12)
    X_G = GDE($\widetilde{X}_G$) // Extract global temporal dependencies using Eqs. (13)–(18)
    X'_G = MSAM(X_G) // Extract multi-scale temporal features using Eqs. (19)–(25)
    X_{fusion} = AFAM(X_L, X'_G) // Adaptive features aggregation using Eqs. (26)–(32)
    X_{relu} = Dropout(ReLU(X_{fusion})) // Activate fused features and apply dropout
    $\hat{Y}$ = FC(X_{relu}) // Fully connected to predict the label for each sample
    model.zero_grad()
    $\mathcal{L}_{loss}$ = CrossEntropyLoss($\hat{Y}, y_{batch}$) // Back-propagate using Eq. (33)
    Back-propagate $\mathcal{L}_{loss}$
    AdamW.step()
  end for
  for batch (X_{batch}, y_{batch}) in validation set **do**
    $\hat{Y}_{val}$ = Model(X_{batch}) // Compute validation loss and accuracy
  end for
  ReduceLROnPlateau.step()
  if early stopping criteria **then**
    Break
  end if
end for
for batch (X_{test}, y_{test}) in testing set **do**
  $\hat{Y}_{test}$ = Model(X_{test}) // Compute test accuracy, macro-F1 and AUROC
end for

Table 1
Time complexity of each module in the DGMANet.

Module	Time complexity
DGM	$O(BN^2)$
GDE	$O(\text{num_layers} \times (BT^2D + BTD^2 + BTDF))$
MSAM	$O(BC^2T)$
AFAM	$O(BC^3 + BC^2T)$

T denotes the sequence length. Temporal attention and residual connections impose negligible additional overhead. Therefore, the overall time complexity of the AFAM module is $O(BC^3 + BC^2\,T)$.

In summary, the primary time complexity of DGMANet is expressed as $O(BN^2) + O(\text{num_layers} \times (BT^2D + BTD^2 + BTDF)) + O(BC^2\,T) + O(BC^3 + BC^2\,T)$. The time complexity of each key module is summarized in Table 1.

4. Experiments analysis

This section evaluates DGMANet on the FW-UAV dataset to verify its diagnostic effectiveness. A series of ablation studies further examine the contributions of each module, and visualization experiments illustrate the strong representation capability of DGMANet.



Fig. 4. Small FW-UAV data acquisition system.

4.1. Experimental setup

The experiments were performed on a Linux server using Python 3.12.9, CUDA 11.8, an NVIDIA RTX A10 GPU with 24 GB GRAM, and PyTorch 2.6.0 as the fault diagnosis framework. The model was trained with a learning rate of 0.00002, weight decay of 0.0005, and a batch size of 4. Each training session consisted of 100 epochs and was repeated for 10 runs to ensure stability and reproducibility. The AdamW optimizer was employed for gradient-based optimization, with `ReduceLROnPlateau` serving as the learning rate scheduler to adjust the rate during training. L2 regularization and early stopping were applied to mitigate overfitting.

4.2. Evaluation metrics

Fault diagnosis assesses a model's ability to detect anomalies within sensor data. In this study, accuracy is the primary metric for performance in few-shot scenarios. It is defined as

$$\text{Accuracy} = \frac{N}{T} \times 100\% \tag{34}$$

where N denotes the number of correctly classified samples, and T represents the total number of samples.

Additionally, Macro-F1 and AUROC are reported as complementary performance metrics. Macro-F1 accounts for class imbalance by computing the unweighted average F1-score across all classes, whereas AUROC evaluates the discriminative performance of the model independent of threshold selection. In multi-class scenarios, the macro-averaged AUROC is reported.

4.3. Data preparation and preprocessing

The real-world UAV flight–fault dataset was collected by Bronz et al. [16] over several days in July 2020. As depicted in Fig. 4, the data acquisition system enabled fault injection and real-time data transmission. In particular, a Ground Control System (GCS) actively injected control-surface faults into a flying FW-UAV during experiments, whereas an RC transmitter managed flight control. The resulting data were transmitted in real time via an X-Bee radio modem to enable fault diagnosis tasks.

Furthermore, to simulate actuator-level faults and assess their impact on flight dynamics, the researchers employed the control-surface fault model proposed in Tandale and Valasek [42], which characterizes faults by control-effectiveness degradation and input bias. The actual control input vector u_{app} is mathematically defined as a function of the desired command input u_{com} and additive disturbances (see Eq. (35)).

$$\begin{bmatrix} u_{app_1} \\ u_{app_2} \end{bmatrix} = \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix} \begin{bmatrix} u_{com_1} \\ u_{com_2} \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \tag{35}$$

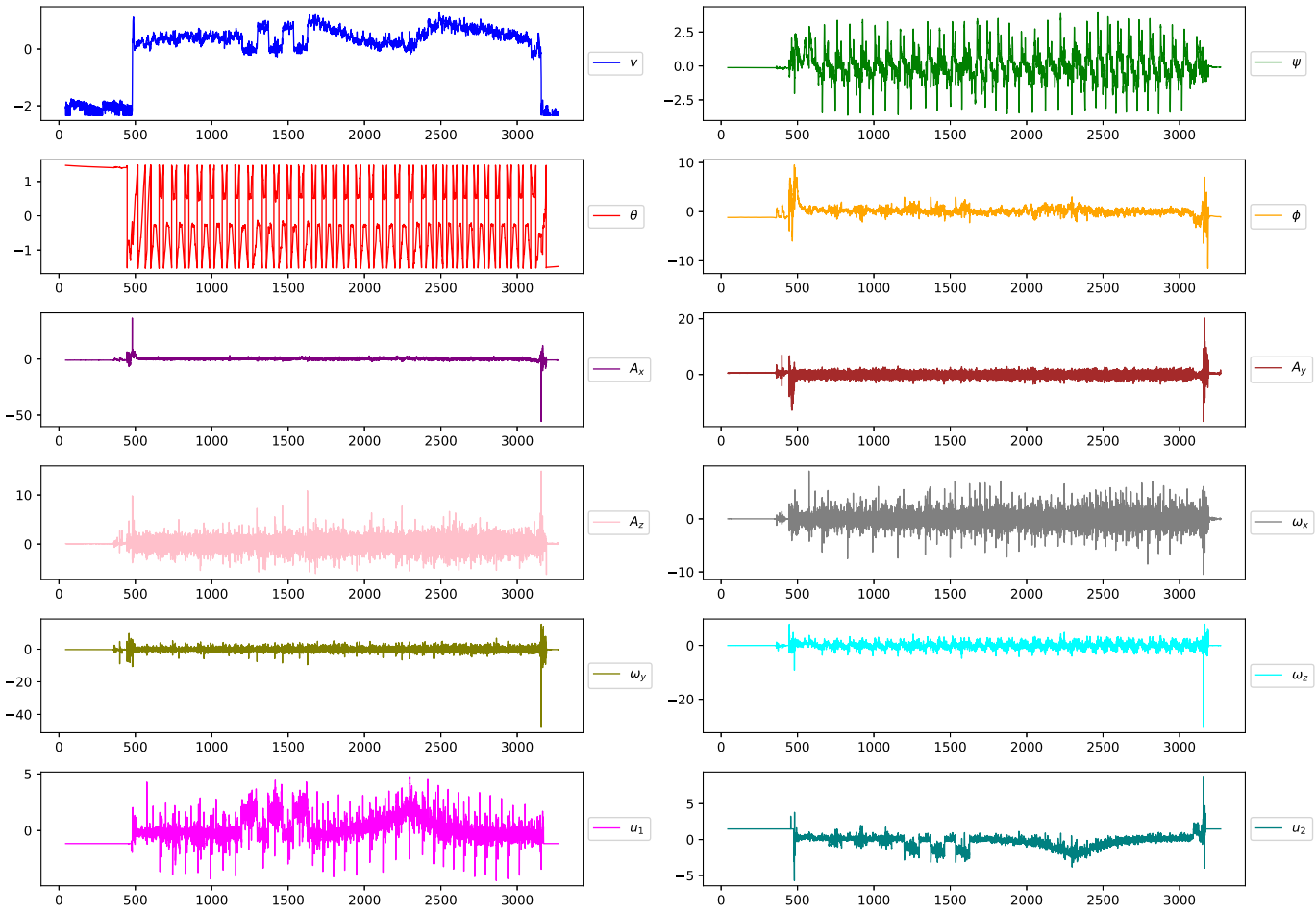


Fig. 5. Normalized July 23 state variables.

In this formulation, u_{app_i} and u_{com_i} denote the actual and commanded inputs for surface i ($i = 1, 2$); d_i (with $0 \le d_i \le 1$) represents the control effectiveness coefficient, and e_i accounts for modeling uncertainty or external disturbances. By varying the diagonal matrix $D = \text{diag}(d_1, d_2)$ and the bias vector $e = [e_1, e_2]^T$, various fault scenarios can be realistically simulated.

The FW-UAV dataset employed in this study comprises over 50 feature variables, thus providing a robust foundation for fault diagnosis tasks. To reduce computational complexity and resource consumption, we selected a representative subset of 12 key variables: $v, \psi, \theta, \phi, a_x, a_y, a_z, \omega_x, \omega_y, \omega_z, u_1, u_2$. In this subset, v denotes airspeed; ψ, θ, and ϕ represent yaw, pitch, and roll angles, respectively; a_x, a_y, and a_z correspond to linear accelerations along the x-, y-, and z-axes; ω_x, ω_y, and ω_z indicate angular rates about the respective body axes; and u_1 and u_2 specify control commands for the right and left control surfaces.

The raw flight data are not immediately applicable for model training, necessitating preprocessing. Initially, owing to inconsistent sampling rates among variables, all signals were resampled to a uniform frequency of 10 Hz via linear interpolation. This ensures time-aligned sequences while preserving the essential temporal structure of the original measurements.

Subsequently, to eliminate scale disparities, we applied Z-score normalization, which standardizes each variable to zero mean and unit variance, as defined below:

$$X = \frac{x - \mu}{\sigma} \tag{36}$$

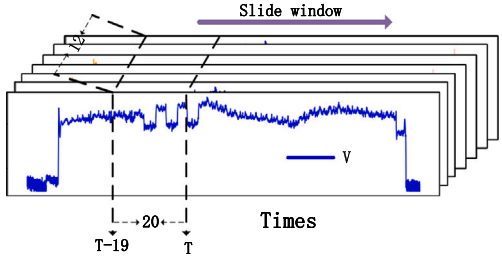


Fig. 6. Data sample production.

where μ denotes the mean and σ the standard deviation of each variable. Fig. 5 illustrates the standardized time-series profiles of 12 representative variables derived from UAV telemetry data collected on July 23, exemplifying the normalization process.

Finally, a sliding window approach generated the training samples along the time axis. As depicted in Fig. 6, at each timestamp t, we extracted a sequence of 20 consecutive observations across the 12 variables, yielding an input tensor $x_t \in \mathbb{R}^{20\times 12}$ as follows:

$$\mathbf{x}_t = \begin{bmatrix} v_t & \cdots & a_{z,t} & \cdots & \omega_{z,t} & \cdots & u_{2,t} \\ v_{t-1} & \cdots & a_{z,t-1} & \cdots & \omega_{z,t-1} & \cdots & u_{2,t-1} \\ \vdots & & \vdots & & \vdots & & \vdots \\ v_{t-19} & \cdots & a_{z,t-19} & \cdots & \omega_{z,t-19} & \cdots & u_{2,t-19} \end{bmatrix} \in \mathbb{R}^{20\times 12} \tag{37}$$

Table 2
Details of the datasets.

Flight date	Wind speed	Classes	Fault type → label	Label → Sample size
July 12	<2.0 m/s	2	Normal → 0, $d_1 = 0.3 \rightarrow 1$	L0 → 4208, L1 → 4772
July 13	8.0 m/s	2	Normal → 0, $d_1 = 0.3 \rightarrow 1$	L0 → 5839, L1 → 3141
July 21	2.5 m/s	9	Normal → 0, $d_1 = 0.3 \rightarrow 1$ $d_2 = 0.9 \sim 0.3 \rightarrow 2 \sim 8$	L0 → 6903, L1 → 2417, L2 → 1473 L3 → 1489, L4 → 1490, L5 → 1555 L6 → 1530, L7 → 1613, L8 → 3508
July 23	5.0 m/s	9	Normal → 0, $d_1 = 0.3 \rightarrow 1$ $d_2 = 0.9 \sim 0.3 \rightarrow 2 \sim 8$	L0 → 6668, L1 → 2525, L2 → 1427 L3 → 1457, L4 → 1479, L5 → 1502 L6 → 1547, L7 → 1616, L8 → 3758

Table 3
Accuracy (%) of each method under small-sample setting.

Methods	July 12	July 13	July 21	July 23	Average
SVM	88.11	84.76	36.96	36.96	61.70
CANN	85.46	84.72	31.87	28.66	57.68
SHNN(5-shot)	89.38	88.36	46.80	41.14	66.42
ASN(5-shot)	88.16	87.37	53.65	42.40	67.90
NDDR CNN-S	79.87	80.66	36.87	38.25	58.91
NDDR CNN-T	80.17	82.21	35.17	33.30	57.71
Cross Stitch-S	79.01	70.40	16.88	27.96	48.56
Cross Stitch-T	91.09	87.25	46.63	47.26	68.06
HDMTL-S	88.64	91.31	66.83	57.37	76.04
HDMTL-T	89.64	91.42	66.65	57.59	76.33
MFFTD-S	88.03	89.24	73.09	64.35	78.68
MFFTD-T	88.10	89.32	73.33	65.38	79.03
DGMANet	**94.50**	**94.24**	**77.77**	**67.19**	**83.43**

4.4. Methods for comparison

We evaluated the proposed DGMANet model for FW-UAV fault diagnosis by comparing it with several representative traditional machine learning and deep learning methods. These methods have demonstrated competitive performance in UAV state recognition and fault diagnosis scenarios.

- **SVM** [16]: A supervised learning algorithm widely used for binary and multi-class classification problems, particularly effective for small-scale datasets.
- **CANN** [43]: A model that integrates spatial and channel attention mechanisms within a CNN to extract diverse fault features, thereby enhancing fault diagnosis performance.
- **SHNN** [18]: A hybrid architecture combining 1D CNN and LSTM to extract spatial and temporal features from flight data, respectively. This approach is specifically designed for small-sample FW–UAV fault diagnosis tasks.
- **ASN** [21]: A twin-tower network comprising two feature extractors with shared weights. The model adaptively reweights channel-wise features to mitigate inter-channel coupling, thus improving performance on limited training data.
- **NDDR-CNN** [44]: This architecture performs discriminative dimensionality reduction via 1D convolution, batch normalization, and weight decay, followed by end-to-end training. It functions as an advanced multi-task learning (MTL) framework.
- **Cross-stitch** [45]: A representative MTL model that employs cross-stitch units to optimize the combination of shared and task-specific representations across convolutional networks.
- **HDMTL** [22]: A heterogeneous MTL framework combining CNN and LSTM to extract spatial and temporal features, respectively. It further employs an Adaptive Sharing Unit (ASU) to integrate complementary knowledge across feature channels, thereby enabling effective spatiotemporal learning for small-sample FW–UAV fault diagnosis.
- **MFFTD** [26]: MFFTD merges spatio-temporal flight data characteristics with dynamic graph structures, employing a mutual information fusion network to address feature heterogeneity.

From a task-type perspective, SVM and CANN serve as classical machine learning and deep learning baselines, respectively, and constitute single-task learning (STL) approaches. SHNN and ASN are meta-learning-based STL methods tailored to small-sample UAV fault diagnosis. However, their reliance on episodic training and support-query matching renders them vulnerable to the scarcity of representative support samples under complex flight conditions. NDDR-CNN, Cross-Stitch, and HDMTL are advanced MTL frameworks that exploit shared spatiotemporal features to enhance generalization and robustness, yet they do not explicitly address the dynamic evolution of spatiotemporal dependencies induced by varying flight regimes. MFFTD represents a dynamic graph-based approach but separates spatial and temporal modeling into independent branches, lacking effective synergy to simultaneously capture transient fault impulses and long-term dynamic drifts. In contrast, DGMANet directly models adaptive spatiotemporal dependencies through an integrated architecture without requiring support sample constraints, thereby achieving targeted and efficient small-sample fault diagnosis for fixed-wing UAVs.

4.5. Performance analysis and evaluation

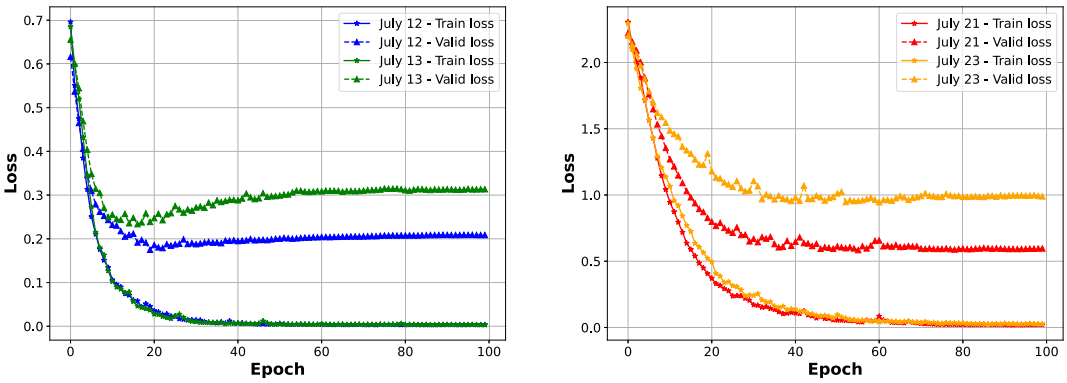
The FW–UAV small-sample fault diagnosis dataset comprises flight data collected on July 12, 13, 21, and 23, as shown in Table 2. Specifically, $d_1 = 0.3$ represents a 30 % efficiency of the right-wing control surface. Similarly, d_2 values of 0.9, 0.8, 0.7, 0.5, 0.4, and 0.3 represent varying degradation levels in the left-wing control surface efficiency. The notation "L0 → 4208" indicates that 4208 samples are labeled as class L0; analogous class labels follow the same format.

4.5.1. Performance analysis under small samples

To evaluate fault diagnosis performance under small-sample conditions, we applied the following evaluation protocol. For each dataset, 90 samples are randomly selected to constitute the training set. To guide training and monitor model convergence, 10 % of the remaining, non-overlapping samples (excluding the 90 training instances) are allocated to a validation set. The model that achieves the highest validation accuracy is retained for final evaluation. Subsequently, 10 % of the entire dataset is designated as the test set to assess the trained model's generalization capability. This procedure yields 898, 898, 2198, and 2198 test samples for the July 12, 13, 21, and 23 datasets, respectively. The remaining data are partitioned following the same procedure. Notably, the training, validation, and test sets are strictly non-overlapping, and the test sets comprise a sufficient number of samples from each class to ensure a representative and balanced evaluation of model performance.

The experimental results of various methods under the 90-sample training condition are summarized in Table 3. In this table, S and T denote the spatial and temporal channels, respectively.

The experimental outcomes illustrate that the proposed DGMANet consistently outperforms all baseline methods. Compared with traditional methods, DGMANet achieved gains of 4.86 and 2.82 percentage points on the binary classification datasets (July 12 and July 13), and surpassed the second-best method, MFFTD, by 4.44 and 1.81 percentage points on the challenging multi-class datasets (July 21 and July 23). Although most methods displayed comparable performance on binary



(a) July 12 and 13 training and validation loss

(b) July 21 and 23 training and validation loss

Fig. 7. Training and validation loss curves for four datasets under 90 samples.

classification tasks, distinctions emerged more clearly in complex multi-class scenarios. Therefore, our analysis focuses on the July 21 and July 23 datasets.

Classical methods such as SVM and CANN performed reasonably on binary tasks but struggled in multi-class settings owing to limited labeled data and poor adaptability to small-sample scenarios. Meta-learning-based approaches such as SHNN and ASN exhibited improved performance under limited data conditions through their episodic training strategies that simulate few-shot learning scenarios. MTL frameworks such as NDDR-CNN and Cross-Stitch did not fully exploit spatiotemporal information owing to their simple feature concatenation or dimensionality reduction strategies. MFFTD, which employs multi-scale feature fusion to capture temporal dynamics at different granularities, achieved the second-best performance with an average accuracy of 79.03 %, illustrating the effectiveness of multi-scale temporal modeling in small-sample scenarios. While HDMTL enhanced feature complementarity through shared representations across spatial and temporal domains, it remained insufficient in capturing spatial topologies and long-range temporal dynamics, achieving an average accuracy of 76.33 %. Neither MFFTD nor HDMTL explicitly models graph-based spatial dependencies, which limits their ability to capture the intricate inter-variable relationships that DGMANet leverages via its graph-structure representation and multi-head attention-based temporal modeling, yielding an average accuracy of 83.43%.

In contrast, DGMANet explicitly represents local spatial topologies and global temporal dependencies within a multi-scale interactive learning framework, which facilitates effective integration of spatiotemporal features. This design markedly enhances its representational capacity and generalization ability. Therefore, DGMANet achieved state-of-the-art performance on all four datasets, demonstrating exceptional effectiveness and robustness in small-sample fault diagnosis for FW–UAV flight scenarios.

As depicted in Fig. 7, training and validation losses steadily decreased for all datasets, indicating satisfactory model convergence. Minor variations emerged in convergence speeds and final loss values among datasets, which reflect inherent data characteristics. The consistent trends between training and validation losses suggest minimal overfitting and confirm strong generalization capability.

In practical UAV operations, diagnostic models are frequently deployed under wind conditions that differ from those encountered during training. To evaluate the robustness of DGMANet and baseline methods under such distribution shifts, cross-condition experiments were conducted. These experiments involved training on July 12 data (wind speed < 2.0 m/s) and testing on July 13 (wind speed ≈ 8.0 m/s), denoted as 12→13, and the reverse scenario (13→12). Additional experiments involved training on July 21 (wind speed ≈ 2.5 m/s) and testing on July 23 (wind speed ≈ 5.0 m/s), designated as 21→23, and the reverse 23→21 configuration. The outcomes are summarized in Table 4.

Table 4
Generalization results for cross-flight conditions.

Methods	July 12→13	July 13→12	July 21→23	July 23→21
SHNN(5-shot)	76.63	83.96	41.80	39.74
ASN(5-shot)	76.26	85.37	48.62	40.21
NDDR CNN-S	69.87	80.13	26.37	36.25
NDDR CNN-T	71.24	79.84	25.94	31.51
HDMTL-S	76.36	82.43	55.23	54.37
HDMTL-T	76.95	82.25	55.35	54.62
MFFTD-S	76.56	84.39	62.39	63.86
MFFTD-T	75.39	84.41	62.58	62.41
DGMANet	**77.25**	**87.79**	**65.54**	**65.48**

A performance asymmetry emerged under distinct wind conditions. In the 21→23 transfer, representing a transition from low to high wind speed in a multi-class fault diagnosis scenario, all models experienced substantial degradation: SHNN and ASN attained only 41.80 % and 48.62 % accuracy, respectively; NDDR-CNN and HDMTL reached 26.37 % and 55.35 %; and MFFTD achieved 62.58 %, demonstrating a marked improvement. Although DGMANet sustained the highest accuracy at 65.54 %, it incurred a 15.7 % relative decline from its original 77.77 % on the July 23 test set. Under the reverse 23→21 condition, corresponding to high-to-low wind speed, performance recovered substantially, with DGMANet declining by only 1.71 percentage points to 65.48%.

A similar pattern appeared in binary classification. In the 13→12 transfer, the models trained under high wind speed assessed under low wind speed. DGMANet achieved 87.79 %, which corresponds to a reduction of 6.9 percentage points from 94.24 % on the July 13 dataset. In contrast, the reverse 12→13 case, transitioning from low to high wind speed, yielded an accuracy of 77.25 %, a relative decrease of 17.25%.

The consistent asymmetry underscores the influence of wind speed variation on model generalization. Training under high-wind conditions exposes models to richer dynamic patterns, including turbulence and strong disturbances, which enables the learning of robust representations that generalize to calmer conditions. Conversely, models trained exclusively under low-wind conditions encounter simpler feature patterns, limiting their capacity to capture high-wind dynamics and resulting in marked performance degradation. These observations align with previous studies on environmental complexity in model generalization and underscore the adaptability of DGMANet under real-world operational variations.

Table 5
Accuracy (%) of fault diagnosis at different training sample sizes on July 12 dataset. **Bold**: Best, Underline: Second.

Methods		Training sample size							
		90	120	150	180	270	360	720	1440
SVM		88.11	89.00	90.00	90.44	92.33	93.56	95.00	97.22
CANN		85.46	88.23	89.65	90.83	93.19	94.52	96.64	98.53
ASN(5-shot)		88.16	91.82	92.68	92.06	95.30	94.86	89.40	92.76
SHNN(5-shot)		89.38	91.31	92.87	93.76	95.82	95.98	97.61	97.04
NDDR-CNN	S	79.87	83.78	88.58	86.87	90.67	93.52	93.84	95.32
	T	80.17	82.71	87.23	85.76	87.30	90.86	91.00	93.80
Cross-stitch	S	79.01	85.51	86.92	89.07	93.54	95.10	97.61	98.90
	T	91.09	89.61	90.16	91.70	95.61	96.68	98.00	99.11
HDMTL	S	88.64	92.03	92.93	92.60	94.49	95.89	96.60	98.61
	T	89.64	92.01	92.85	92.60	94.49	95.86	96.58	98.61
MFFTD	S	88.03	90.38	91.58	92.74	94.71	95.40	97.74	98.92
	T	88.10	90.69	91.66	92.93	94.94	95.50	97.72	98.93
DGMANet		**94.50**	**95.87**	**96.73**	**97.28**	**97.83**	**98.56**	**99.07**	**99.39**
Macro-F1		94.77	96.08	96.90	97.40	97.96	98.64	99.13	99.42
AUROC		98.67	99.07	99.27	99.45	99.63	99.70	99.87	99.92

Table 6
Accuracy (%) of fault diagnosis at different training sample sizes on July 13 dataset. **Bold**: Best, Underline: Second.

Methods		Training sample size							
		90	120	150	180	270	360	720	1440
SVM		84.76	88.88	90.66	93.55	93.88	94.22	95.33	97.00
CANN		84.72	88.15	87.98	91.69	92.74	92.49	97.10	97.62
ASN(5-shot)		87.37	90.30	90.56	91.69	93.39	95.86	97.48	98.63
SHNN(5-shot)		88.36	89.66	91.37	91.35	93.05	96.30	97.80	98.22
NDDR-CNN	S	80.66	82.50	84.75	87.96	89.21	90.53	92.45	93.09
	T	82.21	81.06	83.18	86.13	87.20	88.51	91.43	94.61
Cross-utitch	S	70.40	79.59	83.59	89.58	95.29	95.35	97.93	99.00
	T	87.25	87.20	89.42	92.22	95.63	95.58	97.60	99.02
HDMTL	S	91.31	92.21	92.73	91.13	94.87	95.43	96.91	98.82
	T	91.42	92.14	92.81	91.00	94.81	95.43	96.84	98.85
MFFTD	S	89.24	90.72	91.23	92.35	94.33	95.68	96.44	98.42
	T	89.32	90.72	91.30	92.29	94.41	95.58	96.53	98.36
DGMANet		**94.24**	**95.04**	**95.29**	**96.03**	**97.41**	**97.15**	**98.14**	**99.15**
Macro-F1		91.58	92.94	93.33	94.31	96.33	96.01	98.42	99.42
AUROC		98.22	98.64	98.74	99.00	99.39	99.50	99.70	99.89

4.5.2. Performance analysis under different sample sizes

In the previous section, we validated the effectiveness of DGMANet for fault diagnosis using 90 training instances. Considering that fault data volume can influence detection performance, this section examines the performance of DGMANet across varied training sample sizes. Specifically, we assess whether the model maintains superior performance when trained on larger datasets with more comprehensive fault-related information. To this end, experiments were conducted with training sizes of 90, 120, 150, 180, 270, 360, 720, and 1440 samples. All samples were used for model training, with a non-overlapping 10 % subset reserved for validation and checkpointing. The fault diagnosis results for the July 12, 13, 21, and 23 datasets are presented in Tables 5–8.

Tables 5 and 6 summarize the binary classification performance of various methods on the July 12 and 13 datasets under different training sample sizes. As the results demonstrate, the proposed DGMANet consistently outperforms all baseline methods, achieving 94.50 % and 94.24 % accuracy on the July 12 and 13 datasets with only 90 samples, and improving to 99.39 % and 99.15 % with 1440 samples. Notably, DGMANet attains Macro-F1 scores of 94.77 % and 91.58 % with only 90 samples, along with AUROC values of 98.67 % and 98.22 %, which demonstrates its superior discriminative capability and balanced diagnostic performance across all decision thresholds. This advantage derives from its effective leveraging of spatial topological dependencies among variables and its capture of long-range temporal relationships, rather than solely fusing spatial and temporal features. Among the comparison methods, HDMTL and Cross-Stitch employ multi-task learning strategies that facilitate shared representations across temporal and spatial domains, yielding competitive performance. MFFTD integrates multi-scale feature fusion for temporal dynamics modeling and demonstrates stable performance across both branches. However, these methods do not explicitly model spatial dependencies via graph-based structures, which restricts their ability to capture complex inter-variable relationships as effectively as DGMANet. ASN and SHNN are explicitly designed for few-shot fault diagnosis, explaining their relatively strong performance under limited-data conditions. In contrast, NDDR-CNN, which reduces dimensions by concatenating multi-task features, suffers information loss and underperforms in the low-sample regime. As sample size increases, all models exhibit improved performance because richer feature representations and enhanced fault knowledge arise from larger datasets.

The classification results for the more complex multi-class tasks on the July 21 and 23 datasets are reported in Tables 7 and 8. Across all training sizes, DGMANet consistently outperformed existing methods, achieving 77.77 % and 67.19 % accuracy on the July 21 and 23 datasets, respectively, with 90 samples. This performance surpassed that of the second-best method, MFFTD, by approximately 4–5 percentage points and remained superior as sample sizes increased to 95.19 % and 93.54 % at 1440 samples. DGMANet exhibited exceptional Macro-F1 and AUROC performance, which are critical for multi-class fault diagnosis in UAV systems. On the July 21 dataset with 90 samples, DGMANet obtained a Macro-F1 score of 78.75 % and an AUROC of 97.34 %. The high AUROC values indicate that DGMANet effectively distinguishes multiple fault classes from normal operation. These results demonstrate

Table 7
Accuracy (%) of fault diagnosis at different training sample sizes on July 21 dataset. **Bold**: Best, Underline: Second.

Methods		Training sample size							
		90	120	150	180	270	360	720	1440
SVM		36.96	41.58	47.18	50.64	58.76	63.12	70.48	78.79
CANN		31.87	35.59	37.10	39.38	49.46	54.24	70.13	81.83
ASN(5-shot)		53.65	59.18	62.22	58.49	59.95	60.35	63.22	66.67
SHNN(5-shot)		46.80	55.91	51.77	56.88	58.15	57.96	60.69	60.35
NDDR-CNN	S	36.87	39.53	43.44	46.99	49.60	56.02	63.98	72.61
	T	35.17	36.79	40.17	44.12	46.06	52.73	62.27	72.19
Cross-stitch	S	16.88	19.98	29.36	38.74	63.48	75.34	89.98	93.70
	T	46.63	51.45	55.52	61.13	82.83	84.19	91.90	94.23
HDMTL	S	66.83	70.50	73.53	77.76	82.94	84.06	89.97	92.67
	T	66.65	70.02	73.69	77.63	83.01	83.91	90.04	92.77
MFFTD	S	73.09	77.52	80.26	82.65	84.66	86.73	92.31	95.02
	T	73.33	77.46	80.17	82.62	84.63	86.81	92.36	94.72
DGMANet		**77.77**	**80.82**	**83.05**	**83.59**	**86.97**	**91.06**	**93.72**	**95.19**
Macro-F1		78.75	81.35	83.55	84.94	87.95	89.71	95.09	99.42
AUROC		97.34	97.75	98.09	98.34	98.90	99.25	99.49	99.76

Table 8
Accuracy (%) of fault diagnosis at different training sample sizes on July 23 dataset. **Bold**: Best, Underline: Second.

Methods		Training sample size							
		90	120	150	180	270	360	720	1440
SVM		36.96	41.58	40.92	46.96	54.38	55.18	63.48	74.01
CANN		28.66	31.64	34.29	40.32	44.25	46.44	60.99	74.27
ASN(5-shot)		42.40	47.36	49.76	49.78	51.48	52.06	54.65	55.96
SHNN(5-shot)		41.14	47.50	45.99	46.26	46.60	47.32	48.28	50.14
NDDR-CNN	S	38.25	41.77	42.24	47.64	49.44	52.74	60.87	69.98
	T	33.30	35.93	39.35	42.84	44.69	48.69	60.80	68.98
Cross-stitch	S	27.96	33.39	33.85	44.48	51.25	64.27	84.94	92.47
	T	47.46	50.25	52.49	61.76	69.39	74.89	87.30	93.44
HDMTL	S	57.37	62.71	67.71	70.74	73.56	71.51	83.07	90.20
	T	57.59	62.54	67.61	70.61	73.54	71.64	82.99	90.00
MFFTD	S	64.35	69.63	71.32	72.53	78.76	80.41	86.12	92.78
	T	65.38	69.52	71.41	72.57	78.81	80.36	86.14	92.66
DGMANet		**67.19**	**70.77**	**72.64**	**75.28**	**78.52**	**83.97**	**89.02**	**93.54**
Macro-F1		65.01	69.16	72.02	75.06	79.19	84.28	89.48	94.00
AUROC		94.54	95.94	96.58	97.21	97.92	98.65	99.27	99.66

that DGMANet's explicit modeling of spatial dependencies via graph structures and its capture of long-range temporal dynamics via attention mechanisms significantly enhanced the representational quality and discriminative capacity of the learned features even under challenging classification scenarios. In small-sample settings with 90 to 270 training examples, MFFTD achieved the second-best performance among all baseline methods because of its multi-scale feature fusion strategy, which captures temporal dynamics at multiple granularities, whereas HDMTL produced competitive results by employing a shared learning strategy to enhance complementarity between temporal and spatial features. Nonetheless, both MFFTD and HDMTL lacked explicit graph-based modeling of spatial dependencies, which restricted their ability to capture complex fault patterns embedded in the spatial topology. This highlights the key advantage of DGMANet in overcoming the limitations of conventional methods. As training set sizes increased to 360–1440 samples, MFFTD maintained strong performance and achieved the second-best outcomes in most configurations, whereas HDMTL and Cross-Stitch-T exhibited comparable classification accuracy; nevertheless, DGMANet retained a significant performance edge, indicating that its graph attention mechanism and multi-head attention-based temporal modeling effectively leveraged richer feature information across data regimes. The performance of NDDR-CNN remained constrained by its dimensionality reduction strategy, which may excessively compress or distort salient features. ASN and SHNN, which adopted meta-learning strategies under the "N-way 5-shot" protocol [18,21], were restricted by only five support samples per class, rendering them less effective for complex multi-class tasks. Traditional methods such as SVM and CANN also struggled to learn discriminative representations under limited and heterogeneous data conditions, resulting in unsatisfactory classification performance.

4.5.3. Feature visualization analysis

To comprehensively assess the performance of DGMANet on complex multi-class classification tasks, we conducted deep feature visualization and confusion matrix analysis on the datasets collected on July 21 and July 23. Fig. 8 illustrates the final-layer feature representations for three representative training sample sizes: 90, 270, and 1440. Fig. 9 presents the corresponding confusion matrices, providing an intuitive assessment of classification performance across different data scales.

In the 90-sample scenario, the feature distribution exhibits significant overlap, especially among classes 0, 1, and 2, rendering them nearly indistinguishable. The sample distribution is scattered, which indicates that the model struggles to extract discriminative features owing to the limited training data. However, aside from these three classes, the feature distribution of the remaining categories appears relatively well defined, suggesting that the model effectively differentiates between other fault classes. This observation is reflected in the confusion matrix, which reveals high misclassification rates among classes 0, 1, and 2, whereas classification for other classes is more accurate.

When the number of training samples increases to 270, feature embeddings become more structured and class clusters emerge more clearly. The separation among classes 0, 1, and 2 begins to materialize, and the distinctions among the remaining classes become even more pronounced. The confusion matrix indicates a significant reduction in

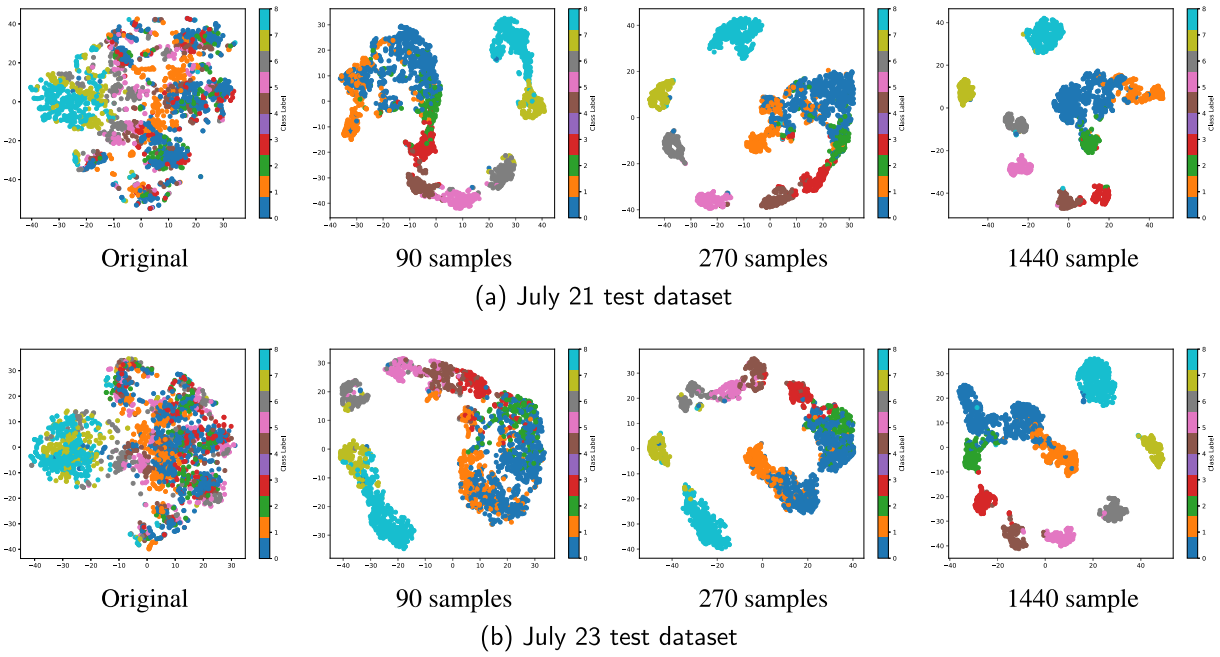


Fig. 8. The t-SNE visualization of the feature representations from the deepest layer in DGMANet on the July 21 and July 23 datasets, showcasing the variations under different sample sizes.

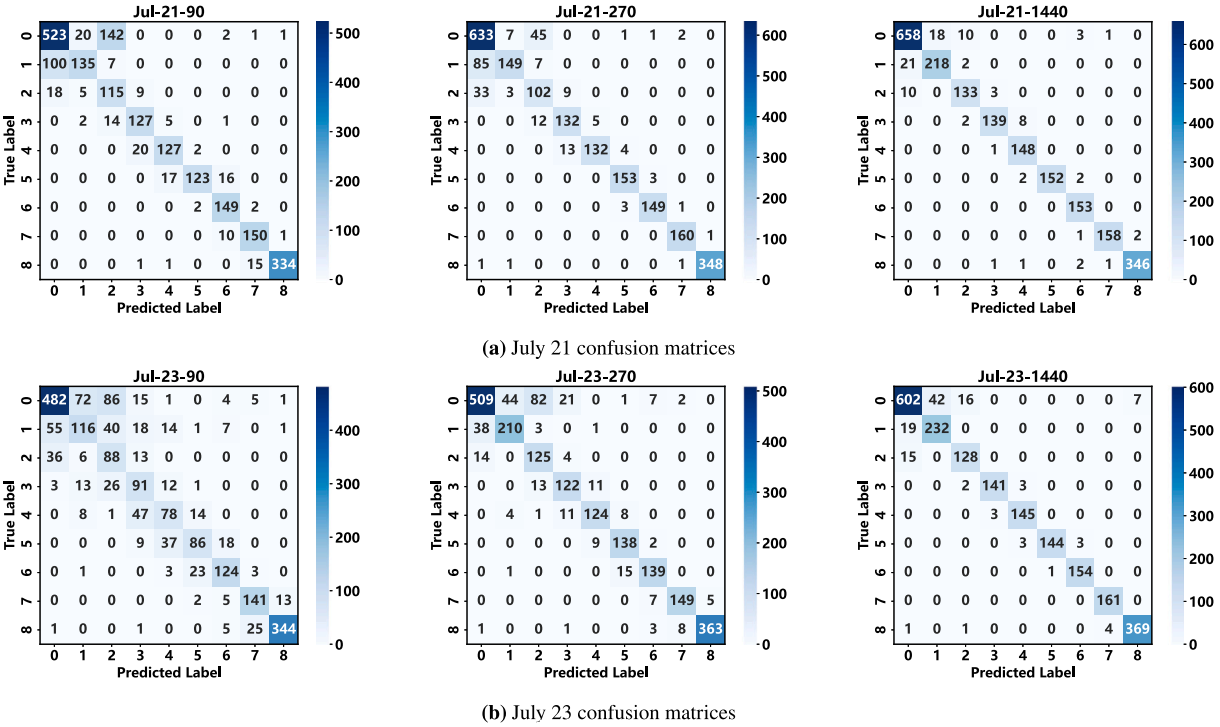
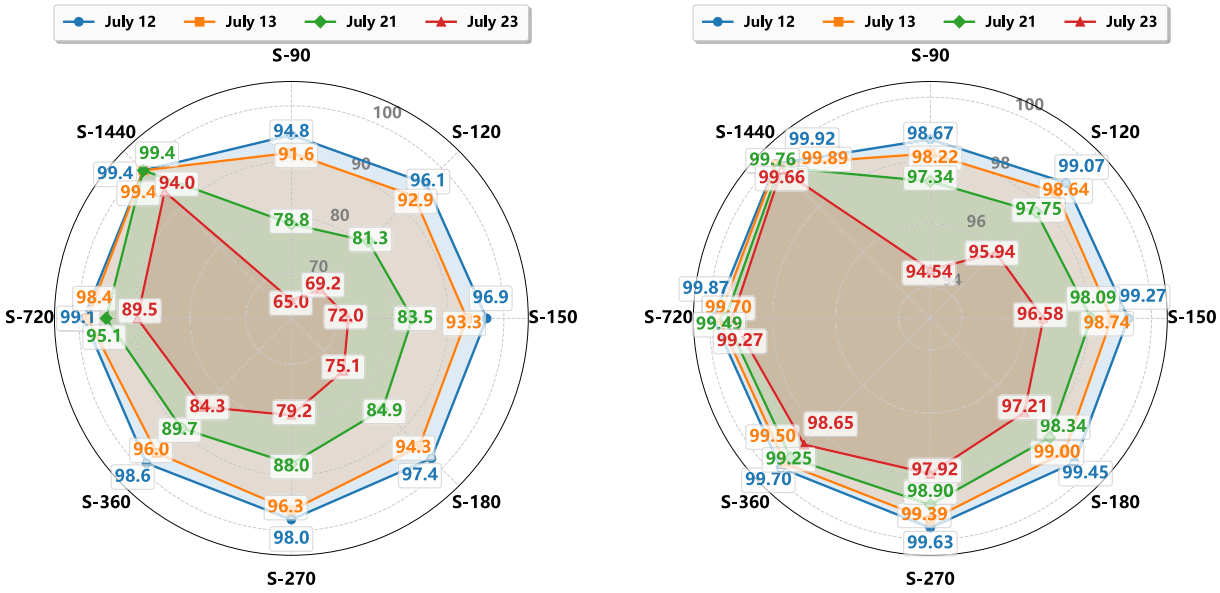


Fig. 9. Confusion matrices comparison between July 21 and July 23 with different training sample.

classification errors and an improvement in overall accuracy, demonstrating that the model learns more reliable and discriminative features as data availability increases.

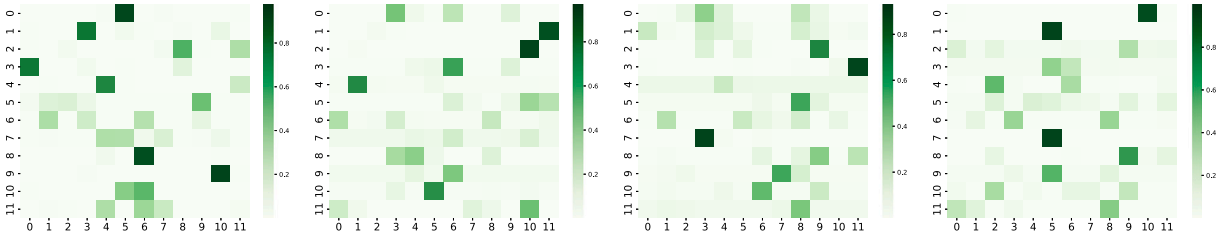
Under the 1440-sample condition, the learned features exhibit strong intra-class compactness and inter-class separability. Each class forms a tightly clustered, well-defined group in the feature space, including the previously ambiguous Classes 0, 1, and 2. The confusion matrix indicates near-perfect classification, demonstrating that DGMANet achieves remarkable generalization and robustness when trained on sufficient data.

With increasing training samples, feature representations evolve from ambiguous and overlapping to compact, well-separated clusters. This progression is evident in the improved capacity of the model to distinguish highly similar and dissimilar classes. The alignment between feature visualizations and confusion matrix analysis substantiates the model's capability to capture heterogeneous topological dependencies



(a) Macro-F1. **(b)** AUROC.

Fig. 10. Radar charts comparing Macro-F1 and AUROC results under small-sample conditions across four datasets. S-90 represents 90 training samples.



(a) July 12 **(b)** July 13 **(c)** July 21 **(d)** July 23

Fig. 11. Visualization of adjacency matrices learned from four datasets after training.

and long-range temporal dynamics for complex multi-class fault diagnosis. Moreover, both Macro-F1 and AUROC confirm the overall performance of the model, as depicted in Fig. 10.

4.5.4. DGM features visualization analysis

The adjacency matrices learned from different datasets are visualized, offering insights into the features captured for each dataset, as illustrated in Fig. 11. These visualizations indicate a sparse structure in the learned adjacency matrices, suggesting that DGMANet focuses on extracting meaningful correlations rather than aggregating superficial variables. For example, they exhibit only a few significant connections, underscoring the ability of the model to identify and preserve key relationships without overfitting. Additionally, the weight distributions of the adjacency matrices vary across datasets. The relationships between variables learned through graph learning appear dataset-dependent and dynamic. The adjacency matrix learned from each dataset exhibits distinct patterns in node connections and weight distributions, confirming the capacity of the model to learn adaptively from the inherent data structure. This observation has practical implications because variable relationships frequently vary with data scale, context, or domain. In summary, the adjacency matrix visualizations in Fig. 11 illustrate the adaptability of the model across datasets by capturing dependency relationships among feature variables, thereby enhancing fault diagnosis performance.

Table 9
Comparison of each model parameters and corresponding diagnostic times.

Methods	Model parameters	Time consumed (s)	Rankings
SVM	198	0.000020	1
CANN	4.87×10^6	0.001689	4
ASN	5.24×10^4	0.038743	8
SHNN	3.85×10^4	0.039216	9
NDDR-CNN	4.32×10^6	0.001170	2
Cross-stitch	2.94×10^6	0.001676	3
HDMTL	1.28×10^7	0.002286	5
MFFTD	1.46×10^7	0.003929	7
DGMANet	1.08×10^7	0.002479	6

4.6. Real-time efficiency analysis

Evaluating real-time performance is essential for assessing the practical utility and computational efficiency of methods. Consequently, this section examines the number of model parameters and diagnosis time to provide a comprehensive understanding of trade-offs among model complexity, diagnostic accuracy, and runtime efficiency. Table 9 presents

Table 10
Ablation experiment results. **Bold**: Best, Underline: Second.

Experiments	Task			
	July 12	July 13	July 21	July 23
w/o DGM	93.13 ± 2.07	93.20 ± 1.62	74.59 ± 3.23	64.05 ± 2.84
w/o AFAM	92.84 ± 1.46	92.06 ± 1.39	72.64 ± 3.41	63.72 ± 3.13
w/o GDE	91.62 ± 1.33	92.74 ± 1.56	64.21 ± 4.28	57.64 ± 4.91
w/o MSAM	93.59 ± 1.40	93.69 ± 1.84	50.82 ± 2.92	46.79 ± 2.78
DGMANet	**94.50 ± 1.69**	**94.24 ± 1.43**	**77.77 ± 3.02**	**67.19 ± 2.58**

total parameter counts and the time required to diagnose a single sample for each method in the July 21 fault diagnosis task.

Regarding parameter scale, MFFTD, HDMTL, and DGMANet exhibit relatively higher architectural complexity than other baseline methods, with parameter counts on the same order of magnitude. However, diagnosis time does not necessarily correlate with model size; for example, although NDDR-CNN contains more parameters, it achieves a notably short diagnosis time. SHNN, despite a lower parameter count, requires the longest diagnosis time owing to its time-intensive meta-testing phase. Among methods with competitive diagnostic accuracy, MFFTD exhibits prolonged diagnosis time because its multi-scale feature fusion strategy processes temporal information at varied granularities, incurring additional computational overhead. HDMTL, by contrast, achieves a more efficient diagnosis time while maintaining similar parameter complexity. All deep learning–based methods satisfy the real-time constraints of FW-UAV sensors; however, their substantial model complexity hinders deployment on resource-limited hardware.

Table 11
Details of hyper-parameters.

Hyper-parameters	Experimental range	Selected value
Batch size	{2,4,8}	4
Learning rate	{0.0001,0.00005,0.00002,0.00001}	0.00002
Dropout	{0.1,0.2,0.3,0.5}	0.3

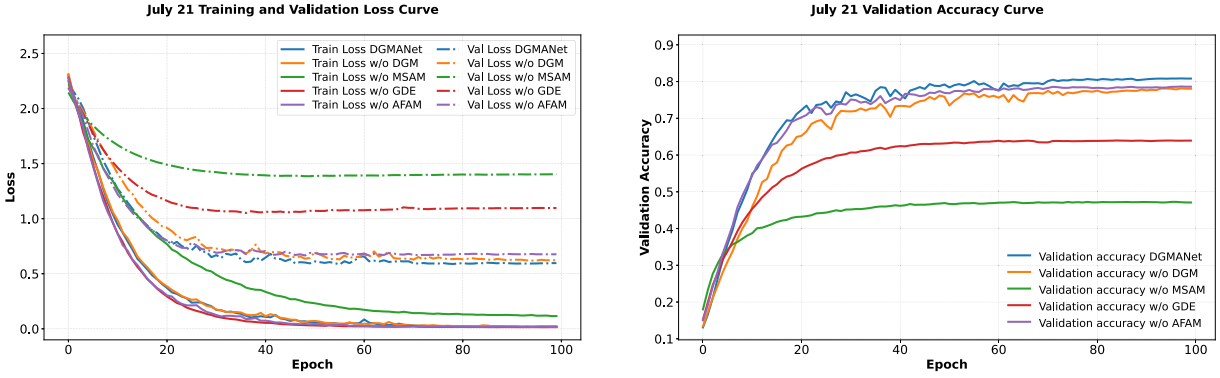
Conversely, the proposed DGMANet model retains competitive parameter efficiency and a favorable diagnosis time while outperforming both MFFTD and HDMTL in diagnostic accuracy. This finding indicates that DGMANet balances predictive precision with computational efficiency, enhancing its practical applicability.

Overall, DGMANet offers a superior trade-off among diagnostic accuracy, model complexity, and runtime efficiency, demonstrating clear advantages in real-time performance. Considering the limited computational resources of GCS, future work will focus on further model compression and optimization to support efficient deployment in real-world UAV systems.

4.7. Ablation study

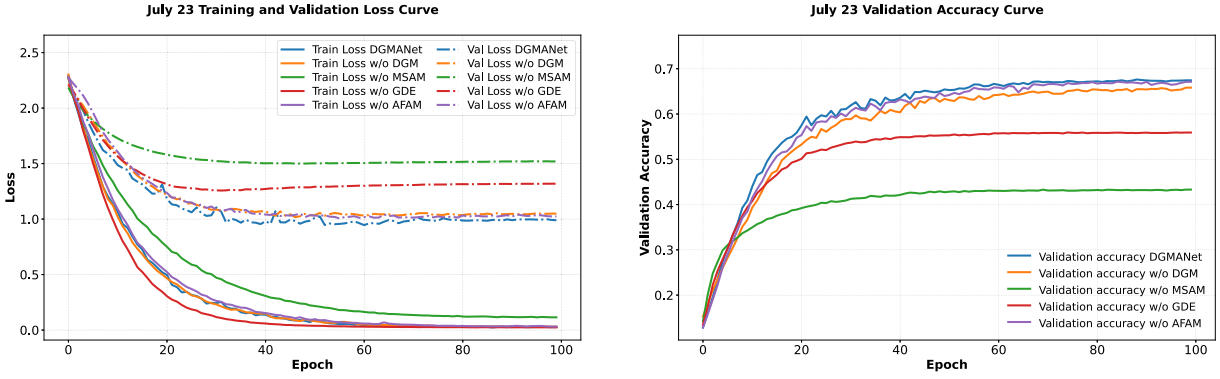
Ablation studies were conducted on four datasets collected on July 12, 13, 21, and 23 (totaling 90 samples) to validate the effectiveness of the key components of DGMANet. The variants of DGMANet with specific components removed are defined as follows:

- **w/o DGM**: DGMANet replaces the dynamic graph-enhanced module with a CNN (with a convolution kernel size of 3).



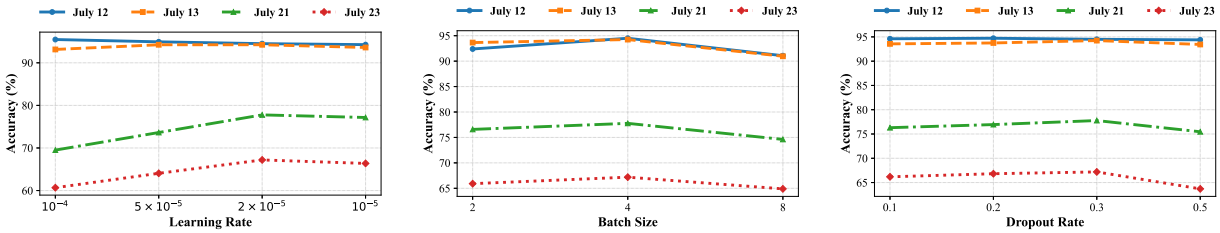
(a) Training and validation loss. **(b)** Validation accuracy.

Fig. 12. Training and validation performance of the proposed model and its ablated variants on the July 21 dataset.



(a) Training and validation loss. **(b)** Validation accuracy.

Fig. 13. Training and validation performance of the DGMANet model and its ablated variants on the July 23 dataset.



(a) Learning rate **(b)** Batch size **(c)** Dropout rate

Fig. 14. Effects of different hyperparameters on fault diagnosis accuracy across four datasets.

- **w/o GDE**: DGMANet excludes the global dependencies extractor. The embedded data is directly input into the MSAM module.
- **w/o MSAM**: DGMANet without the multi-scale attention module. The GDE output is directly fed into AFAM.
- **w/o AFAM**: DGMANet without the adaptive feature aggregation module. Network outputs are fused via weighted summation before the output module.

The experimental results are summarized in Table 10. First, omitting MSAM yielded the largest performance degradation across all tasks. Specifically, accuracy decreased from 77.77 % to 50.82 % on the July 21 dataset and from 67.19 % to 46.79 % on the July 23 dataset. These findings demonstrate that MSAM is vital for capturing multi-scale temporal dependencies, which are essential in complex multi-class classification scenarios.

Second, removing the GDE module resulted in substantial performance declines, particularly on the July 21 and July 23 datasets, indicating that GDE is crucial for modeling global temporal representations and enhancing the ability of the model to distinguish fault patterns with diverse temporal dynamics.

Third, excluding AFAM consistently reduced performance, with notable degradation on the July 12 and July 13 datasets. This finding implies that AFAM enables fine-grained integration of multi-source features, thereby contributing to the accurate identification of temporally stable fault patterns.

Finally, omitting the DGM module impaired the ability of the model to capture spatial dependencies, as reflected by significant accuracy reductions across all tasks. This underscores the importance of dynamic graph-based feature extraction in modeling spatial correlations among sensor variables.

We visualize the training and validation performance of the proposed model and its ablated variants on July 21 and July 23, as illustrated in Figs. 12 and 13. From Fig. 12(a) and Fig. 13(a), the full model consistently achieves lower training and validation losses, which indicates improved convergence and generalization. Conversely, some ablated variants exhibit lower training loss yet poorer validation accuracy, as observed in Fig. 12(b) and Fig. 13(b), suggesting overfitting tendencies when key components are removed. The integration of the DGM, MSAM, GDE, and AFAM modules yields superior performance without increasing training complexity, thereby validating the effectiveness of the design.

Overall, the complete DGMANet model consistently achieved the highest accuracy across all evaluated datasets, underscoring the complementary strengths of each architectural component. Collectively, these results demonstrate the effectiveness and robustness of the proposed architecture in modeling diverse temporal and spatial fault characteristics, emphasizing the necessity of preserving all key modules to fully leverage the representational capacity of the model.

4.8. Hyperparameter sensitivity analysis

In this section, we evaluate the sensitivity of DGMANet to key training hyperparameters, including the learning rate, batch size, and dropout rate, which are critical for both optimization dynamics and generalization. A univariate control strategy was adopted in which each hyperparameter was independently varied within a predefined range while the others were held constant. Table 11 summarizes ranges and optimal values, and Fig. 14 presents results across four datasets.

The experimental results indicate that a learning rate of 2×10^{-5} attains the best balance between convergence speed and generalization. Higher rates (e.g., 1×10^{-4}) result in unstable training and reduced performance on datasets, whereas low rates (e.g., 1×10^{-5}) hinder convergence. Regarding batch size, the model performs best with a value of 4. Although a smaller batch size (e.g., 2) occasionally yields competitive results, it exhibits higher variance and lower robustness; conversely, increasing the batch size to 8 results in degradation owing to reduced gradient noise and less effective regularization.

The dropout rate proves pivotal in controlling overfitting. A moderate value of 0.3 delivers stable, high accuracy across datasets, whereas lower values fail to sufficiently regularize the model and higher values (e.g., 0.5) lead to underfitting and degraded performance. Overall, these findings demonstrate that DGMANet exhibits robust performance across hyperparameter settings, and the optimal configuration reliably transfers to different data distributions.

5. Conclusions

To address the dual challenges of scarce fault samples and complex spatiotemporal dependency modeling in fault diagnosis for FW-UAV under limited data conditions, this paper introduces a novel data-driven framework, DGMANet. However, unlike traditional single-scale feature extraction methods, the proposed model fully exploits the spatiotemporal correlations in UAV flight data by jointly modeling localized topological structures and hierarchical temporal patterns. To capture spatial structure, DGMANet constructs a dynamic graph that learns adaptive inter-variable relationships. A multi-scale attention encoder extracts global temporal dependencies at multiple resolutions. An adaptive feature aggregation module fuses local and global representations, enhancing feature complementarity while suppressing redundancy. In contrast to existing approaches that overlook structural priors or temporal resolution, DGMANet integrates both spatial and temporal cues, producing more expressive and robust feature representations. Extensive experiments on four real-world FW-UAV datasets demonstrated its superior diagnostic performance and robust generalization in small-sample scenarios. However, the model has several limitations. First, its high computational complexity may constrain real-time deployment on resource-limited embedded systems. Second, its generalization across different UAV platforms, sensor configurations, and extreme environmental conditions requires further validation. Moreover, the effectiveness of the model depends on the quality and representativeness of training samples, which might not encompass all rare fault patterns encountered in practice. Future research will prioritize simplifying the model architecture to reduce computational costs while preserving accuracy, incorporating interpretability to enhance transparency in safety-critical applications, and exploring transfer learning approaches

to improve cross-platform generalization with minimal retraining overhead. Furthermore, the proposed approach can be further integrated with advanced sensing technologies and multi-modal data fusion methods to enhance the robustness and real-time performance of the model, thereby more effectively satisfying the highly practical requirements of FW-UAV safe flight missions.

CRediT authorship contribution statement

Rong Yi: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jianguang Lu:** Writing – review & editing, Supervision, Funding acquisition. **Xianghong Tang:** Writing – review & editing.

Funding

This work was supported by Guizhou Provincial Basic Research Program (Natural Science) (QKHJC-ZK(2024)YB062), Guizhou Provincial Key Technology R&D Program (QKHZC(2025)YB146) and Guizhou University Basic Research Program (GDJC(2024)09).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the reviewers for their insightful comments and helpful suggestions.

Data availability

Data will be made available on request.

References

[1] S. Yuan, Y. Li, F. Bao, H. Xu, Y. Yang, Q. Yan, S. Zhong, H. Yin, J. Xu, Z. Huang, et al., Marine environmental monitoring with unmanned vehicle platforms: Present applications and future prospects, Sci. Total Environ. 858 (2023) 159741.
[2] S. H. Ahmed, An analytical study on improving target tracking techniques in wireless sensor networks using deep learning and energy efficiency models, Babylon. J. Netw. 2023 (2023) 100–104.
[3] M. A. Istiak, M.M. M. Syeed, M. S. Hossain, M. F. Uddin, M. Hasan, R. H. Khan, N. S. Azad, Adoption of unmanned aerial vehicle (UAV) imagery in agricultural management: A systematic literature review, Ecol. Inform. 78 (2023) 102305.
[4] Z.T. Al-Qaysi, M. M. Salih, M. L. Shuwandy, M.A. Ahmed, Y. S.M. Altarazi, Multi-tiered cnn model for motor imagery analysis: Enhancing uav control in smart city infrastructure for industry 5.0, Appl. Data Sci. Anal. 2023 (2023) 88–101.
[5] G. K. Fourlas, G. C. Karras, A survey on fault diagnosis and fault-tolerant control methods for unmanned aerial vehicles, Machines 9 (9) (2021) 197.
[6] S. Liang, S. Zhang, Y. Huang, X. Zheng, J. Cheng, S. Wu, Data-driven fault diagnosis of FW-UAVs with consideration of multiple operation conditions, ISA Transactions 126 (2022) 472–485.
[7] Z. Yang, F. Lin, B. M. Chen, Survey of autopilot for multi-rotor unmanned aerial vehicles, in: IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society, IEEE, 2016, pp. 6122–6127.
[8] R. Telford, S. Galloway, Fault classification and diagnostic system for unmanned aerial vehicle electrical networks based on hidden Markov models, IET Electr. Syst. Transp. 5 (3) (2015) 103–111.
[9] R. Sun, Q. Cheng, G. Wang, W. Y. Ochieng, A novel online data-driven algorithm for detecting UAV navigation sensor faults, Sensors 17 (10) (2017) 2243.
[10] M. Saied, T. Mahairy, C. Francis, H. Shraim, H. Mazeh, M. El Rafei, Differential flatness-based approach for sensors and actuators fault diagnosis of a multirotor UAV, IFAC-PapersOnLine 52 (16) (2019) 831–836.
[11] A. Abbaspour, P. Aboutalebi, K. K. Yen, A. Sargolzaei, Neural adaptive observer-based sensor and actuator fault detection in nonlinear systems: Application in UAV, ISA Transactions 67 (2017) 317–329.
[12] J. Zhong, Y. Zhang, J. Wang, C. Luo, Q. Miao, Unmanned aerial vehicle flight data anomaly detection and recovery prediction based on spatio-temporal correlation, IEEE Trans. Rel. 71 (1) (2021) 457–468.
[13] L. A. Al-Haddad, A. A. Jaber, An intelligent fault diagnosis approach for multirotor UAVs based on deep neural network of multi-resolution transform features, Drones 7 (2) (2023) 82.
[14] J. Huang, M. Li, Y. Zhang, L. Mu, Z. Ao, H. Gong, Fault detection and classification for sensor faults of UAV by deep learning and time-frequency analysis, in: 2021 40th Chinese Control Conference (CCC), IEEE, 2021, pp. 4420–4424.
[15] W. Zhang, J. Tong, F. Liao, Y. Zhang, Simulation-to-reality UAV fault diagnosis with deep learning, arXiv preprint arXiv:2302.04410, 2023.
[16] M. Bronz, E. Baskaya, D. Delahaye, S. Puechmore, Real-time fault detection on small fixed-wing UAVs using machine learning, in: 2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC), IEEE, 2020, pp. 1–10.
[17] C. Du, X. Zhang, R. Zhong, F. Li, F. Yu, Y. Rong, Y. Gong, Unmanned aerial vehicle rotor fault diagnosis based on interval sampling reconstruction of vibration signals and a one-dimensional convolutional neural network deep learning method, Meas. Sci. Technol. 33 (6) (2022) 065003.
[18] C. Li, S. Li, A. Zhang, L. Yang, E. Zio, M. Pecht, K. Gryllias, A siamese hybrid neural network framework for few-shot fault diagnosis of fixed-wing unmanned aerial vehicles, J. Comput. Des. Eng. 9 (4) (2022) 1511–1524.
[19] T. Gao, W. Sheng, M. Zhou, B. Fang, L. Zheng, MEMS inertial sensor fault diagnosis using a cnn-based data-driven method, Int. J. Pattern Recognit. Artif. Intell. 34 (14) (2020) 2059048.
[20] J. Song, B. Wu, C. Zhao, W. Shang, Y. Liu, W. Li, Small sample fault diagnosis based on intrinsic characteristic encouraged network for unmanned aerial vehicle sensors, IEEE Trans. Instrum. Meas. (2024).
[21] P. Xiong, Z. Li, Y. Li, S. Huang, C. Liu, F. Gu, Fault diagnosis of UAV based on adaptive siamese network with limited data, IEEE Trans. Instrum. Meas. 72 (2023) 1–11.
[22] Y. Zhang, S. Li, A. Zhang, X. An, FW-UAV fault diagnosis based on knowledge complementary network under small sample, Mech. Syst. Signal Process. 215 (2024) 111418.
[23] Z. Wang, Y. Zheng, S. Liang, H. Zhang, W. Yang, Graph neural network with directional information and global perception fusion for uav fault diagnosis, IEEE Sens. J. (2025).
[24] J. Yao, W. Li, Z. Zheng, X. Jia, Clgnn: Uav fault diagnosis via causal learning and graph neural network, in: 2024 International Joint Conference on Neural Networks (IJCNN), IEEE, 2024, pp. 1–8.
[25] K. He, D. Yu, D. Wang, M. Chai, S. Lei, C. Zhou, Graph attention network-based fault detection for uavs with multivariant time series flight data, IEEE Trans. Instrum. Meas. 71 (2022) 1–13.
[26] G. Li, H. Gui, J. Lu, X. Tang, X. Gao, Multi-scale feature fusion network with temporal dynamic graphs for small-sample fw-uav fault diagnosis, Knowl.-Based Syst. (2025) 114605.
[27] V. Sadhu, S. Zonouz, D. Pompili, On-board deep-learning-based unmanned aerial vehicle fault cause detection and identification, in: 2020 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2020, pp. 5255–5261.
[28] L. Chang, Y.-H. Lin, Meta-learning with adaptive learning rates for few-shot fault diagnosis, IEEE/ASME Trans. Mechatronics 27 (6) (2022) 5948–5958.
[29] H. Deng, Y. Lu, T. Yang, Z. Liu, J. Chen, Unmanned aerial vehicles anomaly detection model based on sensor information fusion and hybrid multimodal neural network, Eng. Appl. Artif. Intell. 132 (2024) 107961.
[30] M. Niepert, M. Ahmed, K. Kutzkov, Learning convolutional neural networks for graphs, in: International Conference on Machine Learning, PMLR, 2016, pp. 2014–2023.
[31] J. Bruna, W. Zaremba, A. Szlam, Y. LeCun, Spectral networks and locally connected networks on graphs, arXiv preprint arXiv:1312.6203, 2013.
[32] Z. Chen, J. Xu, T. Peng, C. Yang, Graph convolutional network-based method for fault diagnosis using a hybrid of measurement and prior knowledge, IEEE Trans. Cybern. 52 (9) (2021) 9157–9169.
[33] B. L.H. Nguyen, T. V. Vu, T.-T. Nguyen, M. Panwar, R. Hovsapian, Spatial-temporal recurrent graph neural networks for fault diagnostics in power distribution systems, IEEE Access 11 (2023) 46039–46050.
[34] P. Yin, J. Nie, X. Liang, S. Yu, C. Wang, W. Nie, X. Ding, A multiscale graph convolutional neural network framework for fault diagnosis of rolling bearing, IEEE Trans. Instrum. Meas. 72 (2023) 1–13.
[35] H. Peng, Z. Mao, B. Jiang, Y. Cheng, Multiscale spatial-temporal bayesian graph conv-transformer based distributed fault diagnosis for UAVs swarm system, IEEE Trans. Aerosp. Electron. Syst. (2024).
[36] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, Y. Xiao, MICN: Multi-scale local and global context modeling for long-term series forecasting, in: The Eleventh International Conference on Learning Representations, 2023.
[37] J. Fu, C. Sun, Z. Yu, L. Liu, A hybrid CNN-LSTM model based actuator fault diagnosis for six-rotor UAVs, in: 2019 Chinese Control and Decision Conference (CCDC), IEEE, 2019, pp. 410–414.
[38] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, arXiv preprint arXiv:2210.02186, 2023.
[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2023).
[40] Y. Mu, M. Shahzad, X. X. Zhu, MPTSNet: Integrating multiscale periodic local patterns and global dependencies for multivariate time series classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, 2025, pp. 19572–19580.

[41] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American chapter of the Association for Computational Linguistics: Human Language Technologies, volume 1 (long and short papers), 2019, pp. 4171–4186.
[42] M. D. Tandale, J. Valasek, Fault-tolerant structured adaptive model inversion control, J. Guid. Control Dyn. 29 (3) (2006) 635–642.
[43] M.-Q. Tran, M.-K. Liu, Q.-V. Tran, T.-K. Nguyen, Effective fault diagnosis based on wavelet and convolutional attention neural network for induction motors, IEEE Trans. Instrum. Meas. 71 (2021) 1–13.
[44] Y. Gao, J. Ma, M. Zhao, W. Liu, A. L. Yuille, NDDR-CNN: Layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 3205–3214.
[45] I. Misra, A. Shrivastava, A. Gupta, M. Hebert, Cross-stitch networks for multi-task learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 3994–4003.